Final Project Report

Kemper Mahanes and Henry Dixon (both did extra credit) 05/01/20

The problem we have selected to solve for the final project is the UGV-UAV cooperative localization problem. The diagram for this system model is shown below in Figure 1. This final report walks through the process of converting the nonlinear combined system to a linearized time-variant system, transition from continuous-time to discrete-time, and Kalman Filter (KF) development and implementation. The Kalman Filter design processes provided focus on two different KF flavors: the Linearized Kalman Filter (LKF) and the Extended Kalman Filter (EKF). Both Kalman Filters are tuned using Truth Model Testing and the performance of the tuned filters are compared.

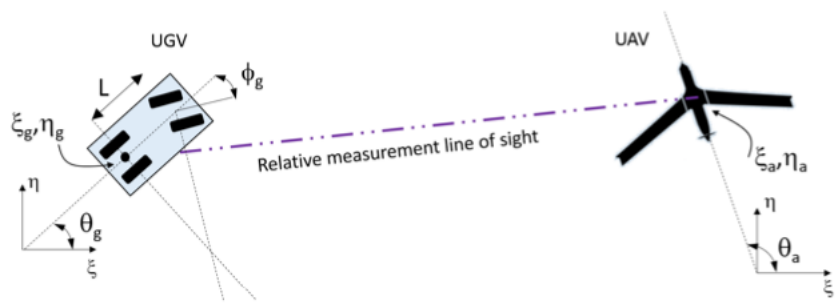

Figure 1: Cooperative air - ground localization problem setup.

1. CT System Jacobians for Linearized Model Parameters

This section defines the the parameters for the cooperative UGV - UAV system, the nonlinear state space definition for the model and shows the derivation of the Jacobians used to linearize the nonlinear system. The dynamics for the UGV system and process noise are modeled by the following equations:

$$\dot{\xi}_g = v_g cos\theta_g + \tilde{w}_{x,g} \tag{1}$$

$$\dot{\eta}_g = v_g sin\theta_g + \tilde{w}_{y,g} \tag{2}$$

$$\dot{\theta}_g = \frac{v_g}{L} tan\phi_g + \tilde{w}_{\omega,g} \tag{3}$$

With control inputs:

$$\boldsymbol{u}_g(t) = \begin{bmatrix} v_g \\ \phi_g \end{bmatrix} \tag{4}$$

And process noise:

$$\tilde{\boldsymbol{w}}_g(t) = \begin{bmatrix} \tilde{w}_{x,g} \\ \tilde{w}_{y,g} \\ \tilde{w}_{\omega,g} \end{bmatrix} \tag{5}$$

In this problem, the UGV begins navigating from a nominal position and initiates a steady continuous turning maneuver. The UGV's wheel base length is $L = 0.5\ m$ and $u_2 \epsilon [-\frac{5\pi}{12}, \frac{5\pi}{12}]\ rad$, and $u_{1,max} = 3\ m/s$.

Likewise, the equations of motion, including process noise, for the UAV are defined as:

$$\dot{\xi}_a = v_a cos\theta_a + \tilde{w}_{x,a} \tag{6}$$

$$\dot{\eta}_a = v_a sin\theta_a + \tilde{w}_{y,a} \tag{7}$$

$$\dot{\theta}_a = \omega_a + \tilde{w}_{\omega,a} \tag{8}$$

With control inputs:

$$\boldsymbol{u}_a(t) = \begin{bmatrix} v_a \\ \omega_a \end{bmatrix} \tag{9}$$

And process noise:

$$\tilde{\boldsymbol{w}}_a(t) = \begin{bmatrix} \tilde{w}_{x,a} \\ \tilde{w}_{y,a} \\ \tilde{w}_{\omega,a} \end{bmatrix} \tag{10}$$

The combined system state, control input, and disturbance input vectors are:

$$\boldsymbol{x}(t) = \begin{bmatrix} \xi_g \\ \eta_g \\ \theta_g \\ \xi_a \\ \eta_a \\ \theta_a \end{bmatrix} \tag{11}$$

$$\boldsymbol{u}(t) = \begin{bmatrix} \boldsymbol{u}_g \\ \boldsymbol{u}_a \end{bmatrix} \tag{12}$$

$$\tilde{\boldsymbol{w}}(t) = \begin{bmatrix} \tilde{\boldsymbol{w}}_g \\ \tilde{\boldsymbol{w}}_a \end{bmatrix} \tag{13}$$

Similar to the UGV, in this problem the aircraft begins flying from an initial position with a nominal trajectory and initiates a steady continuous turning maneuver. The UAV's turning rate is limited to $u_4 \epsilon [-\frac{\pi}{6}, \frac{\pi}{6}]\ rad/s$ and flies with a velocity limited to $u_3 \epsilon [10, 20]\ m/s$.

Lastly, the combined system's sensing model is defined as:

$$\boldsymbol{y}(t) = \begin{bmatrix} tan^{-1}\frac{x_5-x_2}{x_4-x_1} - x_3 \\ \sqrt{(x_1-x_4)^2+(x_2-x_5)^2} \\ tan^{-1}\frac{x_2-x_5}{x_1-x_4} - x_6 \\ x_4 \\ x_5 \end{bmatrix} + \tilde{\boldsymbol{v}}(t) = \boldsymbol{h}(x) + \tilde{\boldsymbol{v}}(t) \tag{14}$$

where $\tilde{\boldsymbol{v}}(t)$ is the AWGN sensor error vector.

Now that the equations of motion for the combined state space system have been defined, we can begin the derivation of the Jacobians needed for linearizing the system. First, the representation of the system in nonlinear state space form is:

$$\dot{\boldsymbol{x}}(t) = \begin{bmatrix} u_1 cosx_3 \\ u_1 sinx_3 \\ \frac{u_1}{L} tanu_2 \\ u_3 cosx_6 \\ u_3 sinx_6 \\ u_4 \end{bmatrix} = \boldsymbol{f}(x,u) + \boldsymbol{\Gamma}\tilde{\boldsymbol{w}}(t) \tag{15}$$

To begin the linearization process, we first need to perform a Taylor Series expansion of $\boldsymbol{f}(x,u)$ and define the mathematics for a new state, $\boldsymbol{\delta}\dot{\boldsymbol{x}}$. The first order expansion of $\boldsymbol{f}(x,u)$ is given by the following:

$$\boldsymbol{\delta}\dot{\boldsymbol{x}}(t) \approx \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}|_{(x_{nom},u_{nom})} \left(\boldsymbol{x}(t) - \boldsymbol{x}_{nom}(t)\right) + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}|_{(x_{nom},u_{nom})} \left(\boldsymbol{u}(t) - \boldsymbol{u}_{nom}(t)\right) \tag{16}$$

And the new state is now defined by:

$$(t) \triangleq \boldsymbol{x}(t) - \boldsymbol{x}_{nom}(t) \;\; and \;\; \boldsymbol{\delta}\dot{\boldsymbol{x}}(t) = \dot{\boldsymbol{x}}(t) - \dot{\boldsymbol{x}}_{nom}(t) \tag{17}$$

$$\boldsymbol{\delta u}(t) = \boldsymbol{u}(t) - \boldsymbol{u}_{nom}(t) \tag{18}$$

$$\boldsymbol{\delta}\dot{\boldsymbol{x}}(t) \approx \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}|_{(x_{nom},u_{nom})} \boldsymbol{\delta x}(t) + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}|_{(x_{nom},u_{nom})} \boldsymbol{\delta u}(t) + \boldsymbol{\Gamma}(t)\tilde{\boldsymbol{w}}(t) \tag{19}$$

Where the new state, $\delta\dot{\boldsymbol{x}}$, represents the combined system dynamics - linearized about perturbations.

The $\boldsymbol{A}$ and $\boldsymbol{B}$ matrices for the state space system are derived from finding the Jacobians of $\boldsymbol{f}(x,u)|_{(x_{nom},u_{nom})}$ with respect to x and u, where $\tilde{\boldsymbol{A}} \triangleq \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}|_{(x_{nom},u_{nom})}$ and $\tilde{\boldsymbol{B}} \triangleq \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}|_{(x_{nom},u_{nom})}$ and x_{nom} and u_{nom} are the a-priori nominal state trajectories and control inputs. The continuous-time Jacobians for linearization and resulting $\tilde{\boldsymbol{A}}$ and $\tilde{\boldsymbol{B}}$ matrices are as follows:

$For\ \ \dot{\boldsymbol{x_1}}:$

$$\frac{\partial \dot{\boldsymbol{x_1}}}{\partial \boldsymbol{x_3}} = -u_1 sinx_3$$

$For\ \ \dot{\boldsymbol{x_2}}:$

$$\frac{\partial \dot{\boldsymbol{x_2}}}{\partial \boldsymbol{x_3}} = u_1 cosx_3$$

$For\ \ \dot{\boldsymbol{x_3}}:$

$$\frac{\partial \dot{\boldsymbol{x_3}}}{\partial \boldsymbol{x}} = \boldsymbol{0}$$

$For\ \ \dot{\boldsymbol{x_4}}:$

$$\frac{\partial \dot{\boldsymbol{x_4}}}{\partial \boldsymbol{x_6}} = -u_3 sinx_6$$

$For\ \ \dot{\boldsymbol{x_5}}:$

$$\frac{\partial \dot{\boldsymbol{x_5}}}{\partial \boldsymbol{x_6}} = u_3 cosx_6$$

$For\ \ \dot{\boldsymbol{x_6}}:$

$$\frac{\partial \dot{\boldsymbol{x_6}}}{\partial \boldsymbol{x}} = \boldsymbol{0}$$

which gives us the following $6x6$ $\tilde{\boldsymbol{A}}$ matrix:

$$\tilde{\boldsymbol{A}} = \begin{bmatrix} 0 & 0 & -u_1 sinx_3 & 0 & 0 & 0 \\ 0 & 0 & u_1 cosx_3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -u_3 sinx_6 \\ 0 & 0 & 0 & 0 & 0 & u_3 cosx_6 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}_{(x_{nom}, u_{nom})} \tag{20}$$

Now, for the $\tilde{\boldsymbol{B}}$ matrix:

$For\ \ \dot{\boldsymbol{x_1}}:$

$$\frac{\partial \dot{\boldsymbol{x_1}}}{\partial \boldsymbol{u_1}} = cosx_3$$

$For\ \ \dot{\boldsymbol{x_2}}:$

$$\frac{\partial \dot{\boldsymbol{x_2}}}{\partial \boldsymbol{u_1}} = sinx_3$$

$For\ \ \dot{\boldsymbol{x_3}}:$

$$\frac{\partial \dot{\boldsymbol{x_3}}}{\partial \boldsymbol{u_1}} = \frac{tanu_2}{L}$$

$$\frac{\partial \dot{\boldsymbol{x_3}}}{\partial \boldsymbol{u_2}} = \frac{u_1}{L} sec^2 u_2$$

$For\ \ \dot{\boldsymbol{x_4}}:$

$$\frac{\partial \dot{\boldsymbol{x_4}}}{\partial \boldsymbol{u_3}} = cosx_6$$

For $\dot{\boldsymbol{x}}_{\mathbf{5}}$:

$$\frac{\partial \dot{\boldsymbol{x}}_{\mathbf{5}}}{\partial \boldsymbol{u}_{\mathbf{3}}} = sinx_6$$

For $\dot{\boldsymbol{x}}_{\mathbf{6}}$:

$$\frac{\partial \dot{\boldsymbol{x}}_{\mathbf{6}}}{\partial \boldsymbol{u}_{\mathbf{4}}} = 1$$

which gives us the following $6x4$ $\tilde{\boldsymbol{B}}$ matrix:

$$\tilde{\boldsymbol{B}} = \begin{bmatrix} cosx_3 & 0 & 0 & 0 \\ sinx_3 & 0 & 0 & 0 \\ \frac{tanu_2}{L} & \frac{u_1}{L}sec^2u_2 & 0 & 0 \\ 0 & 0 & cosx_6 & 0 \\ 0 & 0 & sinx_6 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}_{(x_{nom},u_{nom})} \tag{21}$$

Computing the Jacobians for linearizing the sensing model equations is done in the same manner, except this time with respect to $\boldsymbol{h}(x)$.

For $\boldsymbol{y}_{\mathbf{1}}$:

$$\frac{\partial \boldsymbol{y}_1}{\partial \boldsymbol{x}_1} = \frac{x_5 - x_2}{(\frac{(x_5-x_2)^2}{(x_4-x_1)^2} + 1)(x_4 - x_1)^2}$$

$$\frac{\partial \boldsymbol{y}_1}{\partial \boldsymbol{x}_2} = -\frac{1}{(x_4 - x_1)(\frac{(x_5-x_2)^2}{(x_4-x_1)^2} + 1)}$$

$$\frac{\partial \boldsymbol{y}_1}{\partial \boldsymbol{x}_3} = -1$$

$$\frac{\partial \boldsymbol{y}_1}{\partial \boldsymbol{x}_4} = -\frac{x_5 - x_2}{(x_4 - x_1)^2(\frac{(x_5-x_2)^2}{(x_4-x_1)^2} + 1)}$$

$$\frac{\partial \boldsymbol{y}_1}{\partial \boldsymbol{x}_5} = \frac{1}{(x_4 - x_1)(\frac{(x_5-x_2)^2}{(x_4-x_1)^2} + 1)}$$

For $\boldsymbol{y}_{\mathbf{2}}$:

$$\frac{\partial \boldsymbol{y}_2}{\partial \boldsymbol{x}_1} = \frac{x_1 - x_4}{\sqrt{(x_1 - x_4)^2 + (x_2 - x_5)^2}}$$

$$\frac{\partial \boldsymbol{y}_2}{\partial \boldsymbol{x}_2} = \frac{x_2 - x_5}{\sqrt{(x_1 - x_4)^2 + (x_2 - x_5)^2}}$$

$$\frac{\partial \boldsymbol{y}_2}{\partial \boldsymbol{x}_4} = \frac{x_4 - x_1}{\sqrt{(x_1 - x_4)^2 + (x_2 - x_5)^2}}$$

$$\frac{\partial \boldsymbol{y}_2}{\partial \boldsymbol{x}_5} = \frac{x_5 - x_2}{\sqrt{(x_1 - x_4)^2 + (x_2 - x_5)^2}}$$

For $\boldsymbol{y_3}$:

$$\frac{\partial \boldsymbol{y}_3}{\partial \boldsymbol{x}_1} = -\frac{x_2 - x_5}{(x_1 - x_4)^2(\frac{(x_2-x_5)^2}{(x_1-x_4)^2} + 1)}$$

$$\frac{\partial \boldsymbol{y}_3}{\partial \boldsymbol{x}_2} = \frac{1}{(x_1 - x_4)(\frac{(x_2-x_5)^2}{(x_1-x_4)^2} + 1)}$$

$$\frac{\partial \boldsymbol{y}_3}{\partial \boldsymbol{x}_4} = \frac{x_2 - x_5}{(\frac{(x_2-x_5)^2}{(x_1-x_4)^2} + 1)(x_1 - x_4)^2}$$

$$\frac{\partial \boldsymbol{y}_3}{\partial \boldsymbol{x}_5} = -\frac{1}{(x_1 - x_4)(\frac{(x_2-x_5)^2}{(x_1-x_4)^2} + 1)}$$

$$\frac{\partial \boldsymbol{y}_3}{\partial \boldsymbol{x}_6} = -1$$

For $\boldsymbol{y_4}$:

$$\frac{\partial \boldsymbol{y}_4}{\partial \boldsymbol{x}_4} = 1$$

For $\boldsymbol{y_5}$:

$$\frac{\partial \boldsymbol{y}_5}{\partial \boldsymbol{x}_5} = 1$$

Using the computed partial derivatives, we can now construct the linearized $\tilde{\boldsymbol{C}}$ Matrix, where $\boldsymbol{C} \triangleq \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}|_{(x_{nom})}$. As well, the new state $\delta \boldsymbol{y}$ can be defined to represent the sensor dynamics - linearized about perturbations to the system.

$$\tilde{\boldsymbol{C}} = \begin{bmatrix} \frac{x_5-x_2}{(\frac{(x_5-x_2)^2}{(x_4-x_1)^2}+1)(x_4-x_1)^2} & -\frac{1}{(x_4-x_1)(\frac{(x_5-x_2)^2}{(x_4-x_1)^2}+1)} & -1 & -\frac{x_5-x_2}{(x_4-x_1)^2(\frac{(x_5-x_2)^2}{(x_4-x_1)^2}+1)} & \frac{1}{(x_4-x_1)(\frac{(x_5-x_2)^2}{(x_4-x_1)^2}+1)} & 0 \\ \frac{x_1-x_4}{\sqrt{(x_1-x_4)^2+(x_2-x_5)^2}} & \frac{x_2-x_5}{\sqrt{(x_1-x_4)^2+(x_2-x_5)^2}} & 0 & \frac{x_4-x_1}{\sqrt{(x_1-x_4)^2+(x_2-x_5)^2}} & \frac{x_5-x_2}{\sqrt{(x_1-x_4)^2+(x_2-x_5)^2}} & 0 \\ -\frac{x_2-x_5}{(x_1-x_4)^2(\frac{(x_2-x_5)^2}{(x_1-x_4)^2}+1)} & \frac{1}{(x_1-x_4)(\frac{(x_2-x_5)^2}{(x_1-x_4)^2}+1)} & 0 & \frac{x_2-x_5}{(\frac{(x_2-x_5)^2}{(x_1-x_4)^2}+1)(x_1-x_4)^2} & -\frac{1}{(x_1-x_4)(\frac{(x_2-x_5)^2}{(x_1-x_4)^2}+1)} & -1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \tag{22}$$

$$\boldsymbol{\delta y}(t) \approx \tilde{\boldsymbol{C}}|_{x_{nom}} \, \boldsymbol{\delta x}(t) + \tilde{\boldsymbol{v}}(t) \tag{23}$$

The derived Jacobians for our continuous-time can now be used to redefine the state space equations for our linearized time-varying (LTV) system. The new state space system of equations is as follows:

$$(\dot{\boldsymbol{x}}_{nom} + \boldsymbol{\delta}\dot{\boldsymbol{x}}) = \boldsymbol{f}(\boldsymbol{x}_{nom} + \boldsymbol{\delta x}, \boldsymbol{u}_{nom} + \boldsymbol{\delta u}) + \boldsymbol{\Gamma}(t)\tilde{\boldsymbol{w}}(t) \tag{24}$$

$$\boldsymbol{y}(t) = \boldsymbol{h}(\boldsymbol{x}_{nom} + \boldsymbol{\delta x}) + \tilde{\boldsymbol{v}}(t) \tag{25}$$

Where the full state is:

$$\boldsymbol{x}(t) = \boldsymbol{x}_{nom}(t) + \boldsymbol{\delta x}(t) \tag{26}$$

The full forcing input is:

$$\boldsymbol{u}(t) = \boldsymbol{u}_{nom}(t) + \boldsymbol{\delta u}(t) \tag{27}$$

Perturbations from $\boldsymbol{x}_{nom}$ are:

$$\boldsymbol{\delta x}(t) = \boldsymbol{x}(t) + \boldsymbol{x}_{nom}(t) \tag{28}$$

And perturbations from $\boldsymbol{u}_{nom}$ are:

$$\boldsymbol{\delta u}(t) = \boldsymbol{u}(t) + \boldsymbol{u}_{nom}(t) \tag{29}$$

2. DT Linearized System

Now that the continuous-time Jacobians and linearized system matrices have been computed, we can form the linearized discrete-time system matrices. However, the discretization process will differ from how we have performed this in the past for LTI systems, because our combined model is a Linear Time-Varying system. Because the system is time varying observability and controllability analysis will not be done. Instead of forming the augmented matrix $\hat{\boldsymbol{A}}$ and computing the matrix exponential $\boldsymbol{e}^{(\hat{\boldsymbol{A}}\Delta t)}$, we will calculate $\tilde{\boldsymbol{F}}$, $\tilde{\boldsymbol{G}}$, and $\tilde{\boldsymbol{H}}$ from $\tilde{\boldsymbol{A}}$, $\tilde{\boldsymbol{B}}$, and $\tilde{\boldsymbol{C}}$ at each time step. This is done by using the CT Jacobians derived in Section 1 and a-priori nominal state values (calculated beforehand/offline) from $\boldsymbol{f}(x_{nom}, u_{nom})$ and $\boldsymbol{h}(x_{nom}, u_{nom})$ to update $\tilde{\boldsymbol{A}}$, $\tilde{\boldsymbol{B}}$, and $\tilde{\boldsymbol{C}}$. After the matrix update operation, the discretized matrices are calculated by:

$$\tilde{\boldsymbol{F}}_{k+1} = \boldsymbol{I} + \Delta T \tilde{\boldsymbol{A}} \tag{30}$$

$$\tilde{\boldsymbol{G}}_{k+1} = \Delta T \tilde{\boldsymbol{B}} \tag{31}$$

$$\tilde{\boldsymbol{H}}_{k+1} = \tilde{\boldsymbol{C}} \tag{32}$$

The time step size provided in the problem statement for model dicretization is $\Delta T = 0.1\ s$, and after discretizing the continuous-time Jacobians (evaluated at the nominal trajectory conditions) the DT system of equations for the combined state space model are:

$$\boldsymbol{\delta x}[k+1] = \tilde{\boldsymbol{F}}_k|_{nom[k]}\, \boldsymbol{\delta x}[k] + \tilde{\boldsymbol{G}}_k|_{nom[k]}\, \boldsymbol{\delta u}[k] + \tilde{\boldsymbol{\Omega}}_k \boldsymbol{w}[k] \tag{33}$$

$$\boldsymbol{\delta y}[k+1] = \tilde{\boldsymbol{H}}_k|_{nom[k+1]}\, \boldsymbol{\delta x}[k+1] + \boldsymbol{v}[k+1] \tag{34}$$

And the total state equations are:

$$\boldsymbol{x}[k+1] = \boldsymbol{x}_{nom}[k+1] + \boldsymbol{\delta x}[k+1] \tag{35}$$

$$\boldsymbol{y}[k+1] = \boldsymbol{y}_{nom}[k+1] + \boldsymbol{\delta y}[k+1] \tag{36}$$

3. Nonlinear and Linearized Model Simulation Results

The nonlinear combined system, defined in the first section of this report, was simulated for 100 seconds using the *ode*45 function in *MATLAB* and a nominal state perturbation. Simulations of the nonlinear and linearized DT systems were performed neglecting process and measurement noise inputs. The initial state perturbation vector applied in these simulations was made to be the same as the one provided for the project reference plots - to ensure the accuracy of our linearized system. This perturbation vector was defined as:

$$\boldsymbol{\delta x} = [0,\ 1,\ 0,\ 0,\ 0,\ 0.1]^T$$

The actual full system responses are plotted in Figure 2 - with the nominal responses overlayed.

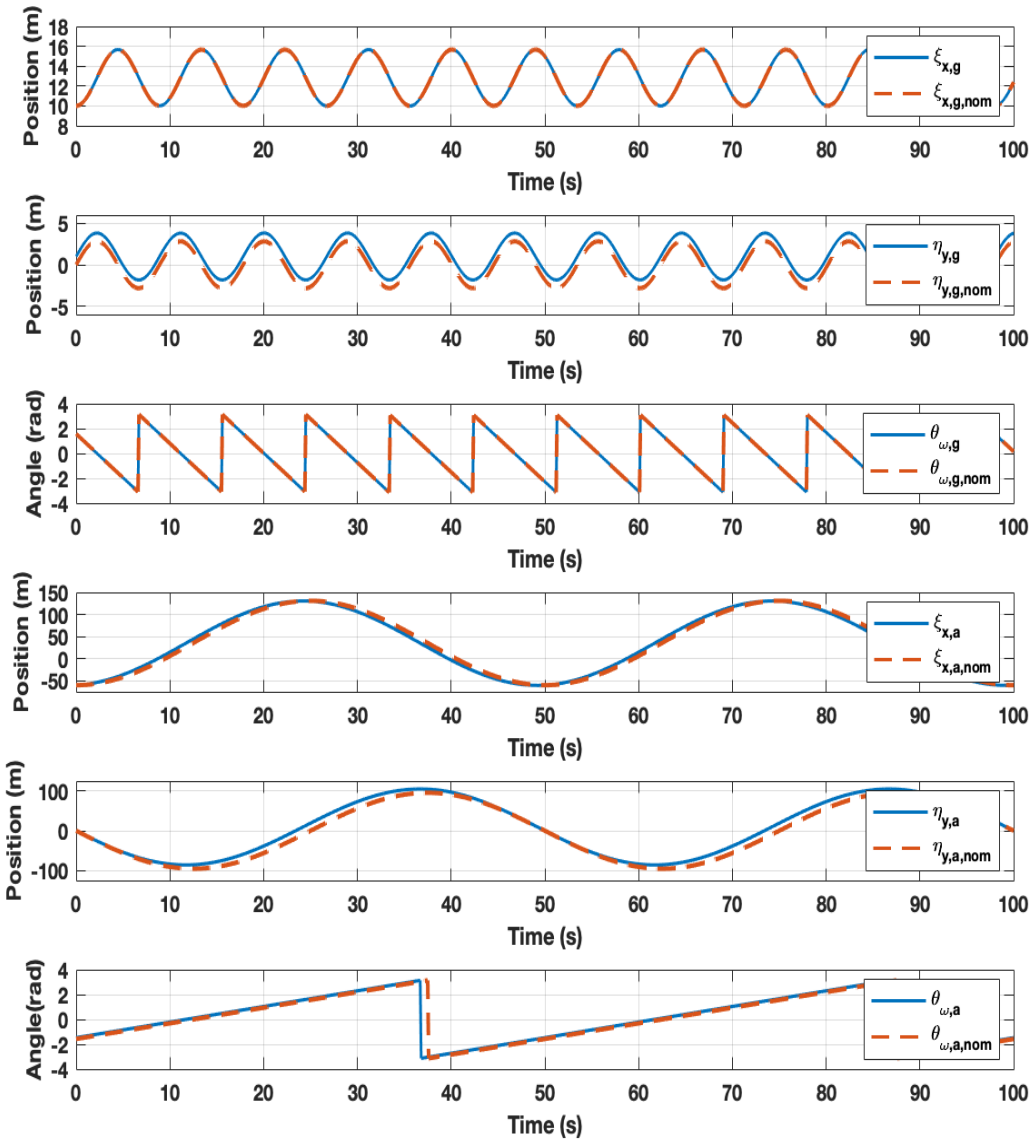

Figure 2: The nonlinear model simulation results show both vehicles' trajectories from nominal inputs and actual positions due to perturbation inputs.

Departure in state value agreement between the nominal and actual is easily observable in the above plots, where the initial state perturbation alters the nonlinear system from its nominal trajectory. Now, comparing those results with the following plots in Figure 3, we will see how our derived linearized time-varying model corrects state errors when the system departs from nominal trajectories.

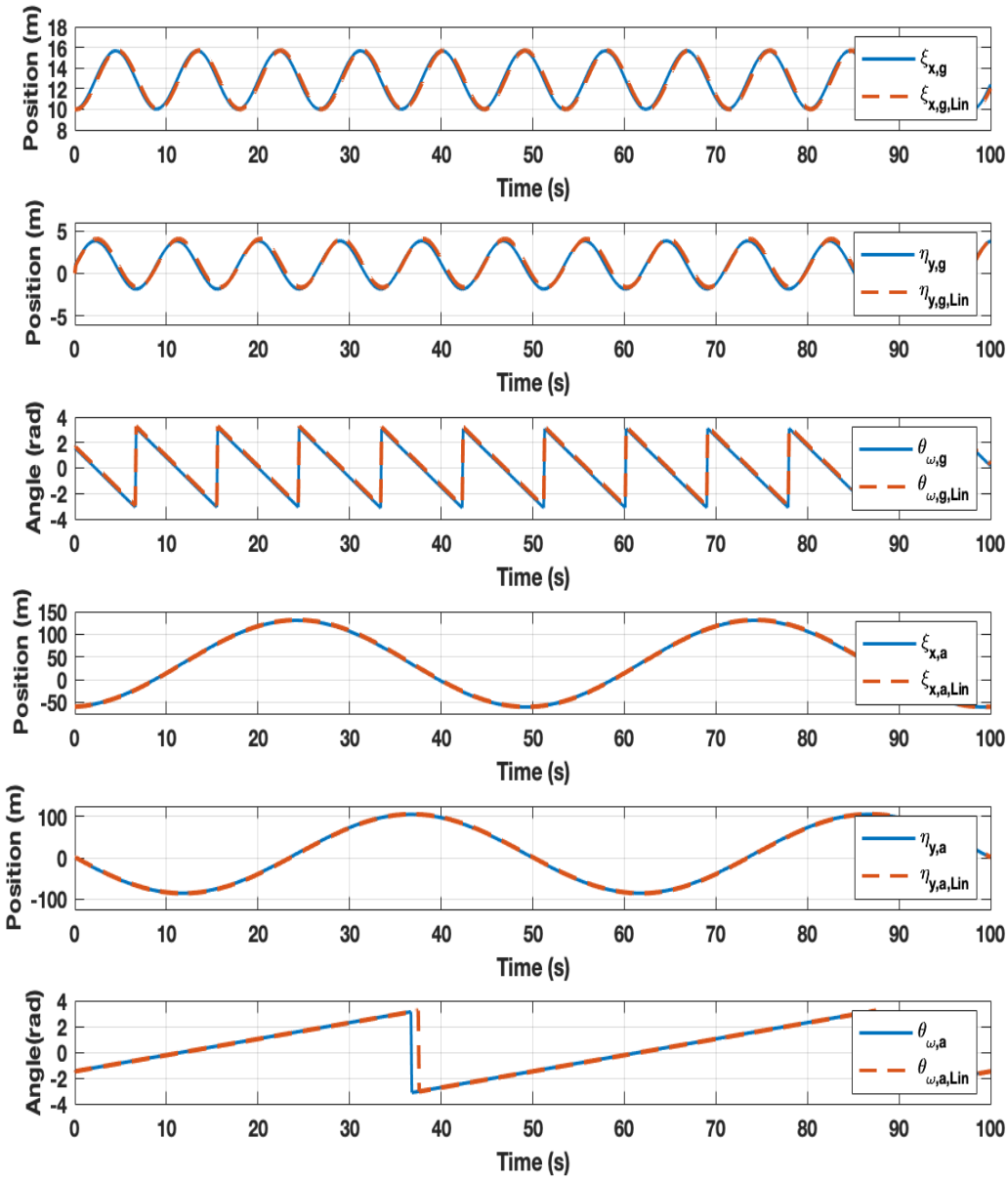

Figure 3: The results from the derived LTV model simulation show much less error between states when the system is perturbed from its nominal trajectory.

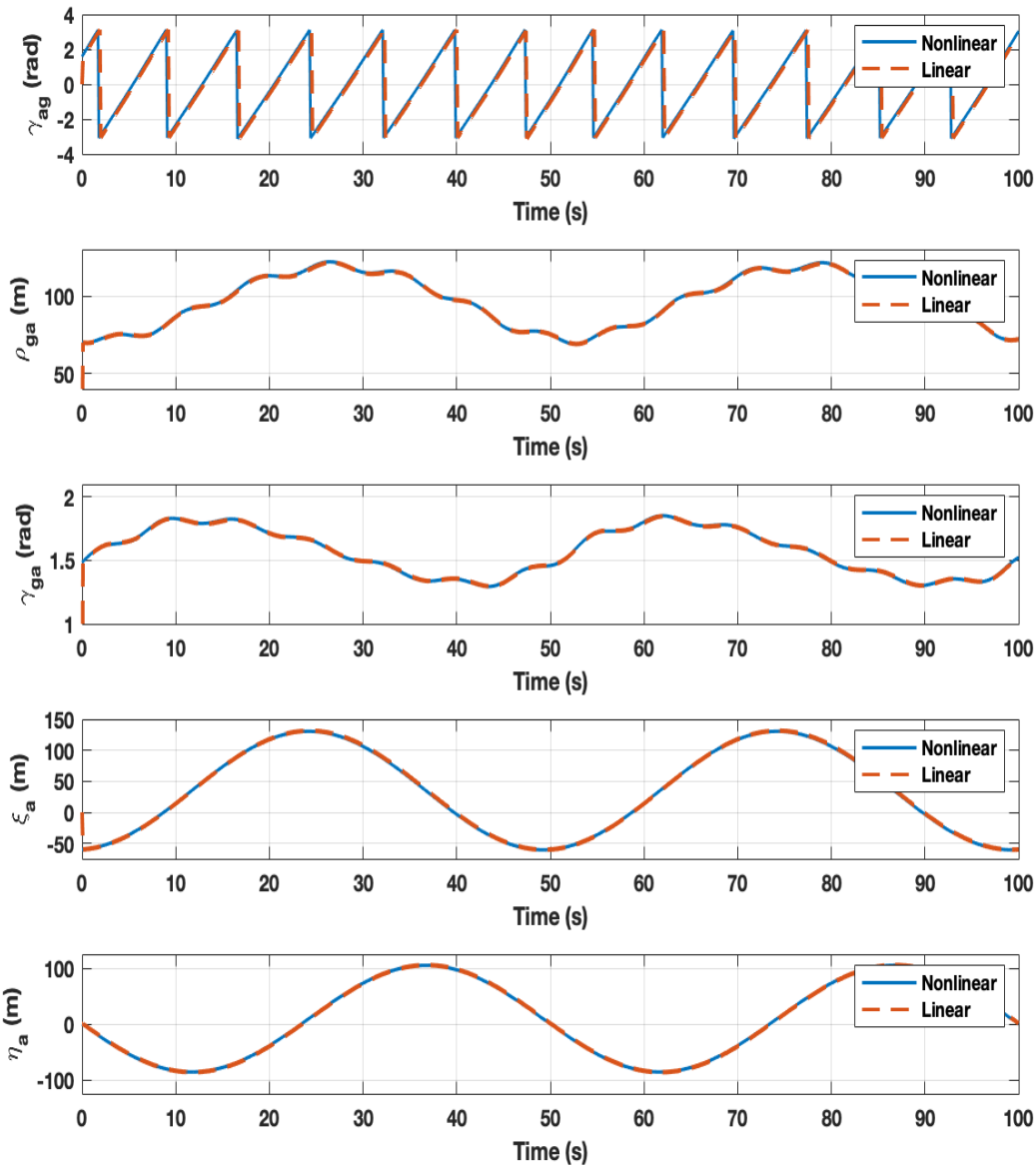

Figure 4: Compares the nonlinear sensor output data with linearized sensor output.

The above results show that the linear approximation for the output data does a good job of following the true nonlinear output. Output 1, 2 and 3 (γ_{ag}, ρ_{ga}, γ_{ga}) follow closely even though the sensor dynamics were nonlinear. Outputs 4 and 5 (ξ_a, η_a) follow exactly given that they already had a linear relationship.

4. Kalman Filter Algorithm Derivations

Now that we have shown our LTV combined system model has been linearized correctly, we can begin the process of designing and implementing Kalman Filters to improve state estimation accuracy when process and sensor noise are present in the system. Both filter implementations will make use of the continuous-time Jacobian matrices derived earlier. However, the main difference between the LKF and EKF is that the LKF uses a nominal trajectory calculated beforehand and only estimates state perturbations at each time step; while the EKF estimates the total state online and uses the most recent estimate to update the filter and total state estimate.

4.1 Linearized Kalman Filter (LKF) Algorithm

Development of the Linearized Kalman Filter algorithm contains two main calculation stages: the prediction step, and the measurement-update correction step. The total state is estimated as $\hat{\boldsymbol{x}}^+_{k+1} \approx \boldsymbol{x}^*_{k+1} + \delta\hat{\boldsymbol{x}}^+_{k+1}$ - where $\boldsymbol{x}^*_{k+1}$ is the offline nominal state trajectory at $t = t_{k+1}$. However, before the filter can begin to run, it must be initialized with a guess for the error uncertainty covariance matrix (P_0).

The first stage begins with the continuous-time perturbation dynamics model for the combined system and the nominal trajectory (calculated offline for all time steps).

$$\boldsymbol{\delta\dot{x}}(t) \approx \tilde{\boldsymbol{A}}|_{(x_{nom},u_{nom})}\,\boldsymbol{\delta x}(t) + \tilde{\boldsymbol{B}}|_{(x_{nom},u_{nom})}\,\boldsymbol{\delta u}(t) + \boldsymbol{\Gamma}(t)\tilde{\boldsymbol{w}}(t)$$

$$\boldsymbol{\delta y}(t) \approx \tilde{\boldsymbol{C}}|_{(x_{nom})}\,\boldsymbol{\delta x}(t) + \tilde{\boldsymbol{v}}(t)$$

The continuous-time Jacobian matrices are evaluated at $t = t_k$ and then discretized to convert $\tilde{\boldsymbol{A}}$ and $\tilde{\boldsymbol{B}}$ to $\tilde{\boldsymbol{F}}_k$ and $\tilde{\boldsymbol{G}}_k$ (using Equations 30 and 31). Then, the prediction updates are calculated as follows for $t = t_{k+1}$:

$$\boldsymbol{\delta u}_k = \boldsymbol{u}_k - \boldsymbol{u}^*_k \tag{37}$$

$$\boldsymbol{\delta\hat{x}}^-_{k+1} = \tilde{\boldsymbol{F}}_k\,\boldsymbol{\delta\hat{x}}^+_k + \tilde{\boldsymbol{G}}_k\,\delta\boldsymbol{u}_k \tag{38}$$

$$\boldsymbol{P}^-_{k+1} = \tilde{\boldsymbol{F}}_k + \boldsymbol{P}^+_k\tilde{\boldsymbol{F}}^T_k + \tilde{\boldsymbol{\Omega}}_k\tilde{\boldsymbol{Q}}_k\tilde{\boldsymbol{\Omega}}^T_k \tag{39}$$

Next, the measurement correction step can begin after the $\tilde{\boldsymbol{H}}_{k+1}$ is updated using $\boldsymbol{x}^*_{k+1}$. This step updates the Kalman Gain $\boldsymbol{K}_{k+1}$ and filter error covariance matrix, then adjusts the state perturbation estimate for $t = t_{k+1}$. The correction step is calculated as follows:

$$\boldsymbol{\delta y}_{k+1} = \boldsymbol{y}_{k+1} - \boldsymbol{y}^*_{k+1} \tag{40}$$

$$\boldsymbol{K}_{k+1} = \boldsymbol{P}^-_{k+1}\tilde{\boldsymbol{H}}^T_{k+1}(\tilde{\boldsymbol{H}}_{k+1}\boldsymbol{P}^-_{k+1}\tilde{\boldsymbol{H}}^T_{k+1} + \boldsymbol{R}_{k+1})^{-1} \tag{41}$$

$$\boldsymbol{P}_{k+1}^{+} = (\boldsymbol{I} - \boldsymbol{K}_{k+1}\tilde{\boldsymbol{H}}_{k+1})\boldsymbol{P}_{k+1}^{-} \tag{42}$$

$$\boldsymbol{\delta}\hat{\boldsymbol{x}}_{k+1}^{+} = \boldsymbol{\delta}\hat{\boldsymbol{x}}_{k+1}^{-} + \boldsymbol{K}_{k+1}(\boldsymbol{\delta y}_{k+1} - \tilde{\boldsymbol{H}}_{k+1}\,\boldsymbol{\delta}\hat{\boldsymbol{x}}_{k+1}^{-}) \tag{43}$$

The filter operates recursively, so after the measurement-update correction step is complete the filter returns to the beginning of the prediction step calculations at the next time step increment.

4.2 Extended Kalman Filter (EKF) Algorithm

The algorithm procedure for the EKF is almost identical to the LKF algorithm. However, instead of using a pre-calculated, fixed, nominal trajectory it uses the most recent total state estimate to linearize the combined system about. This feature of the EKF can help to improve estimation accuracy since the filter is updating the entire state estimate at every time step, instead of just the state perturbation estimates. The EKF also has two stages, and before the filter begins it must be initialized with a guess for the initial total state and error covariance matrix $\hat{\boldsymbol{x}}^{+}(0)$ and $\hat{\boldsymbol{P}}^{+}(0)$. After initialization, the prediction step starts by computing the updated online total state estimate and is used to update the discretized Jacobian matrices. The calculations are as follows:

$$\hat{\boldsymbol{x}}_{k+1}^{-} = \boldsymbol{f}(\hat{\boldsymbol{x}}_{k}^{+}, \boldsymbol{u}_k, \boldsymbol{w}_k = 0) \tag{44}$$

Then $\tilde{\boldsymbol{F}}_k$ is updated as:

$$\tilde{\boldsymbol{F}}_k|_{\hat{x}_k^{+}, u_k, w_k=0} = \boldsymbol{I} + \Delta\boldsymbol{T} \cdot \tilde{\boldsymbol{A}}|_{(\hat{x}_k^{+}, u(t_k), w(t_k)=0)} \tag{45}$$

$$\boldsymbol{P}_{k+1}^{-} = \tilde{\boldsymbol{F}}_k\boldsymbol{P}_k^{+}\tilde{\boldsymbol{F}}_k^{T} + \tilde{\boldsymbol{\Omega}}_k\boldsymbol{Q}_k\tilde{\boldsymbol{\Omega}}_k^{T} \tag{46}$$

Like the LKF algorithm, after the prediction step is complete, the measurement-update correction step begins with a live online update for $\hat{\boldsymbol{y}}_{k+1}^{-}$ and $\tilde{\boldsymbol{H}}_{k+1}$ using the most recent prediction for $\hat{\boldsymbol{x}}_{k+1}^{-}$. The measurement update step calculations are as follows:

$$\hat{\boldsymbol{y}}_{k+1}^{-} = \boldsymbol{h}(\hat{\boldsymbol{x}}_{k+1}^{-}, \boldsymbol{v}_{k+1} = 0) \tag{47}$$

$$\tilde{\boldsymbol{H}}_{k+1} = \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}|_{\hat{x}_{k+1}^{-}} \tag{48}$$

$$\tilde{\boldsymbol{e}}_{y_{k+1}} = \boldsymbol{y}_{k+1} - \hat{\boldsymbol{y}}_{k+1}^{-} \tag{49}$$

$\tilde{\boldsymbol{e}}_{y_{k+1}}$ is the updated error measurement from the difference of the actual sensor data and predicted sensor output.

$$\tilde{\boldsymbol{K}}_{k+1} = \boldsymbol{P}^{-}_{k+1}\tilde{\boldsymbol{H}}^{T}_{k+1}(\tilde{\boldsymbol{H}}_{k+1}\boldsymbol{P}^{-}_{k+1}\tilde{\boldsymbol{H}}^{T}_{k+1} + \boldsymbol{R}_{k+1})^{-1} \tag{50}$$

$$\hat{\boldsymbol{x}}^{+}_{k+1} = \hat{\boldsymbol{x}}^{-}_{k+1} + \tilde{\boldsymbol{K}}_{k+1}\tilde{\boldsymbol{e}}_{y_{k+1}} \tag{51}$$

$$\boldsymbol{P}^{+}_{k+1} = (\boldsymbol{I} - \tilde{\boldsymbol{K}}_{k+1}\tilde{\boldsymbol{H}}_{k+1})\boldsymbol{P}^{-}_{k+1} \tag{52}$$

Where $\hat{\boldsymbol{x}}^{+}_{k+1}$ is the corrected total state estimate and $\boldsymbol{P}^{+}_{k+1}$ is the corrected covariance matrix for $t = t_{k+1}$. The EKF algorithm also operates recursively, so at the end of the measurement update step the filter returns to the beginning of the prediction step.

5. Linearized & Extended Kalman Filter Simulation

With both algorithms defined, we can now begin the implementation process for both filters. We have been provided with sensor data and covariance matrices for the process and measurement noise in the system. The process noise covariance matrix $\boldsymbol{Q}_{true}$ was used to generate ground truth data for $\boldsymbol{x}$. The LKF was initialized with $\boldsymbol{\delta}\hat{\boldsymbol{x}}^{+}(0) = [0,\ 0,\ 0,\ 0,\ 0,\ 0]^{T}$ and $\hat{\boldsymbol{P}}^{+}_{0} = 9 \cdot diag([2,\ 2,\ 2,\ 10,\ 10,\ 2])$. Figure 5 and 6 below show the simulation results for our initial LKF design.

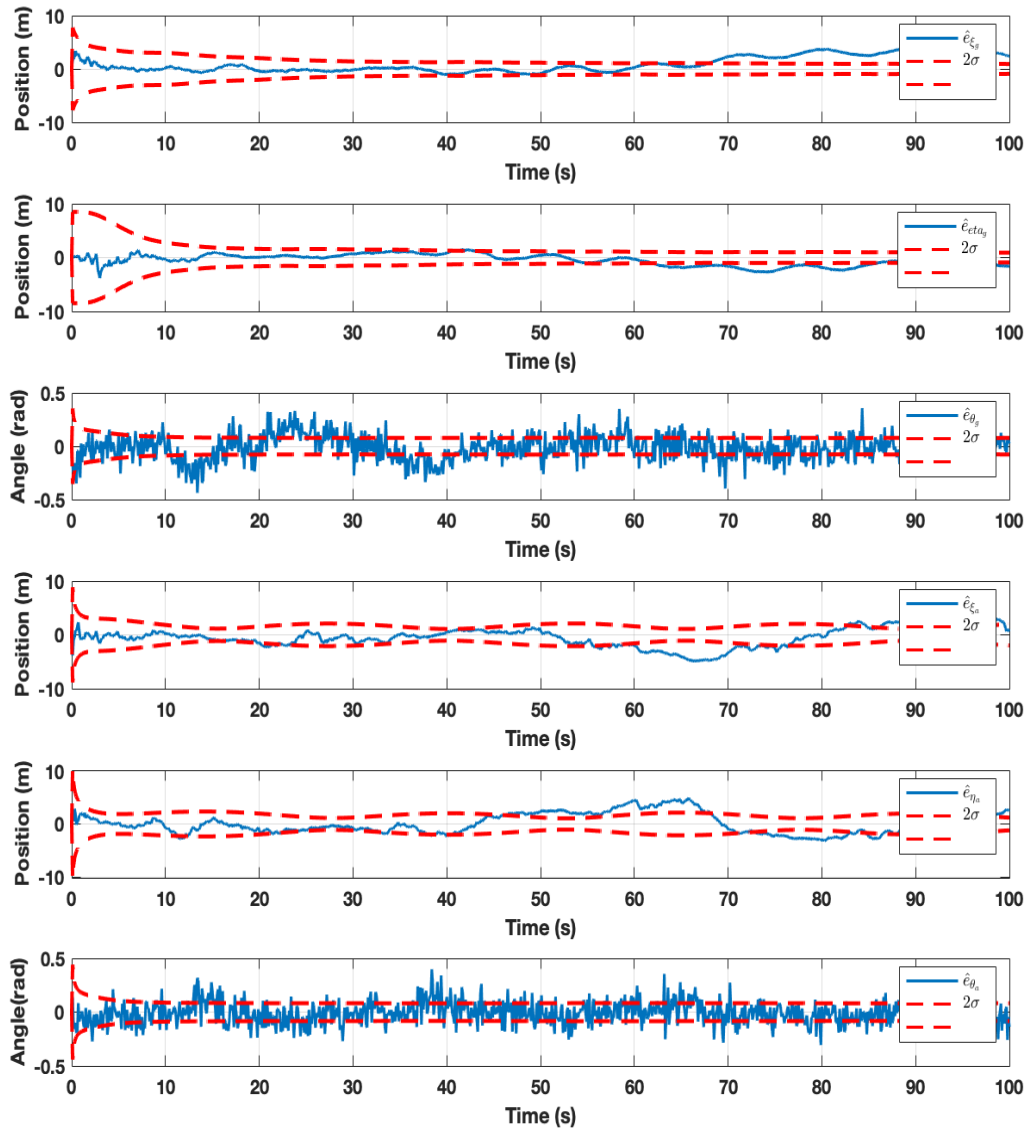

Figure 5: LKF 2σ bounds and filter errors for all six states over 100 seconds.

Observing the filter error results from simulation of the initial LKF design shows that the estimate errors for each state stay within the 2σ bounds for stretches, but periodically go on excursions outside the bounds. When these errors grow too large the LKF can run up against its limitations and the filter can lose tracking ability.

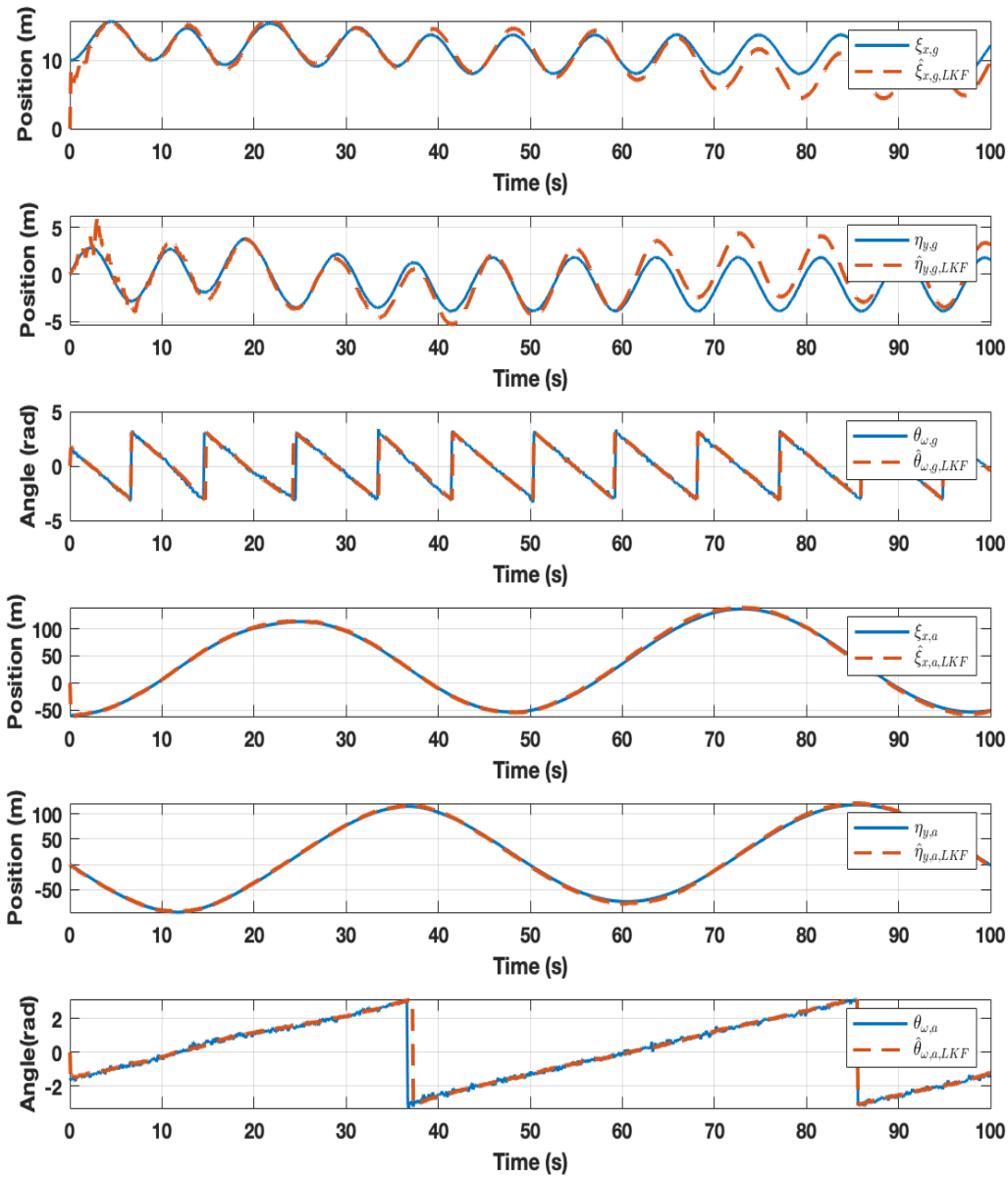

Figure 6: LKF estimate tracking compared to ground truth data.

Here we can observe that when filter errors grow too large for certain states, the filter estimate tracking significantly degrades.

Following the initial implementation, Monte-Carlo simulations were performed to compute the NEES and NIS statistics to tune the filter and assess its performance (discussed in the next section). After the tuning process was completed, the tuned Linearized Kalman Filter was simulated again and results from the refined filter simulation are shown in Figure 7 and Figure 8.

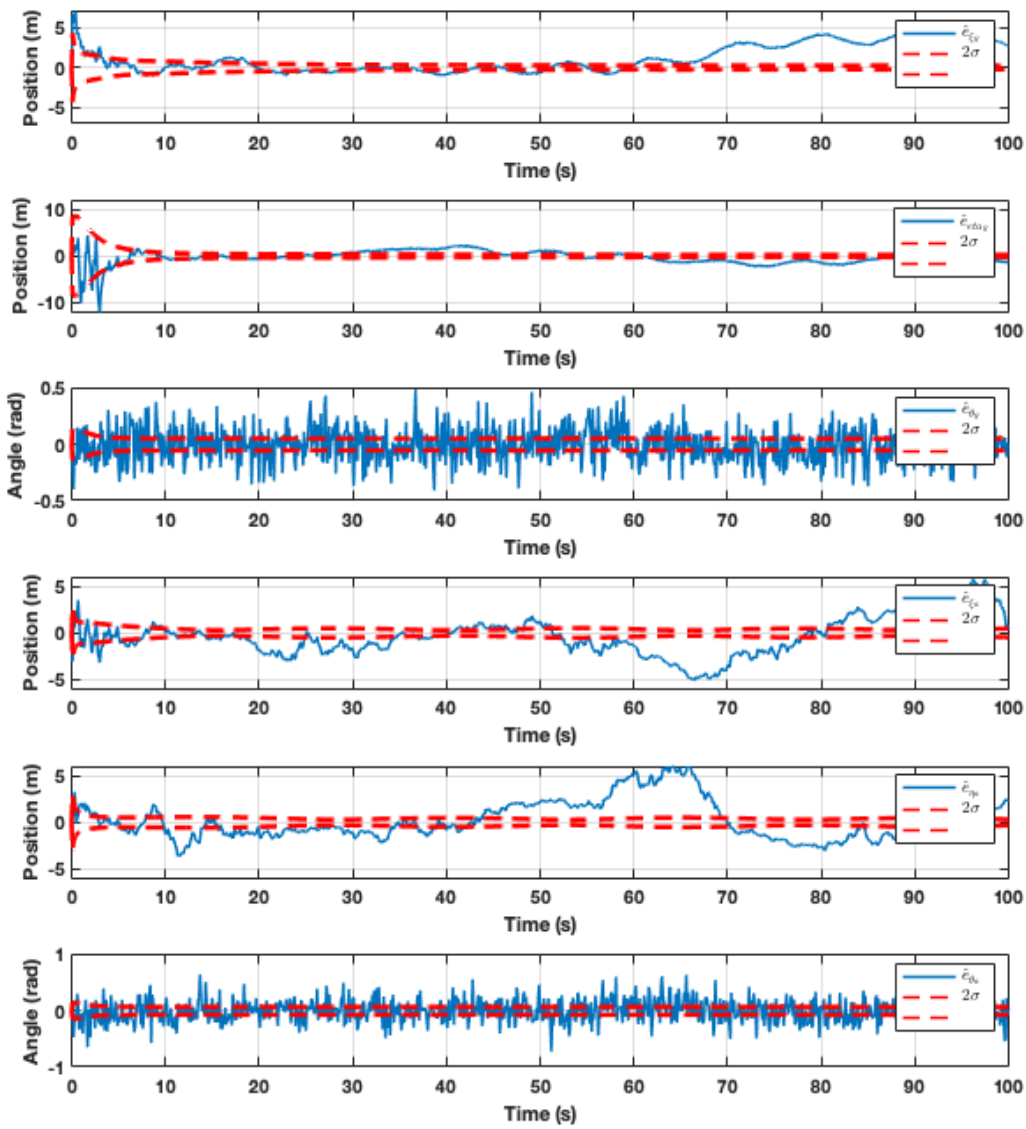

Figure 7: Tuned LKF estimator error and 2σ bounds.

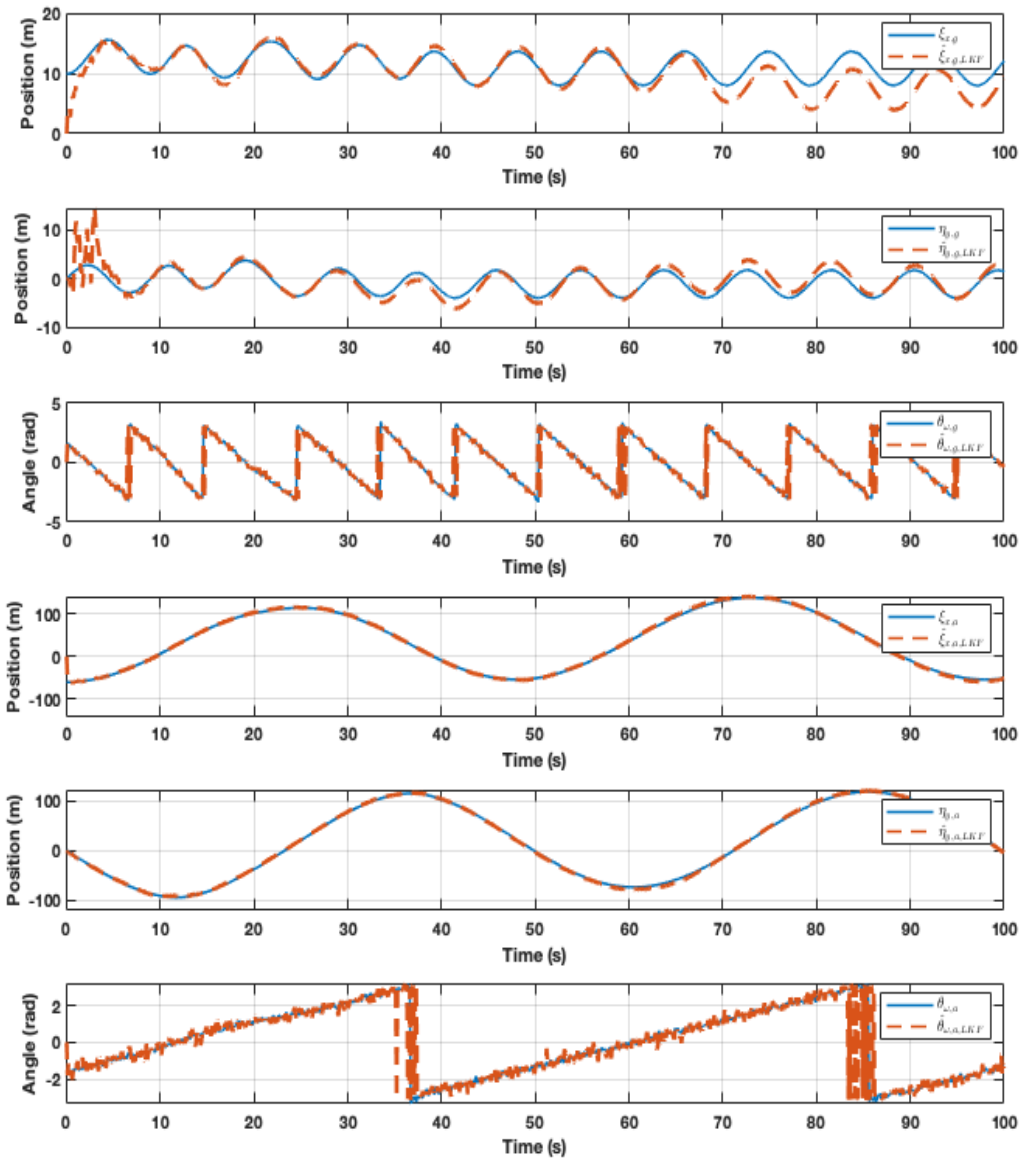

Figure 8: Tuned LKF estimator output compared to ground truth.

The simulation results from the tuned LKF show marginal improvement in state estimate error. Even though the filter's predicted stability and performance was significantly improved in the tuning process, the filter still runs up against its limitations when state perturbations are too large. This is why we did not see a major reduction in estimate error.

Next, we'll see how the Extended Kalman Filter can provide us with better estimate performance than the LKF. The filter was initialized with total state and error covariance matrix $\hat{\boldsymbol{x}}^{+}(0) = \boldsymbol{x}^{*}(0) = \boldsymbol{\mu}_x(0)$ and $\hat{\boldsymbol{P}}_0^{+} = 9\, diag([2,\ 2,\ 2,\ 10,\ 10,\ 2])$. Figures 9 and 10 show the results from the initial EKF design implementation.

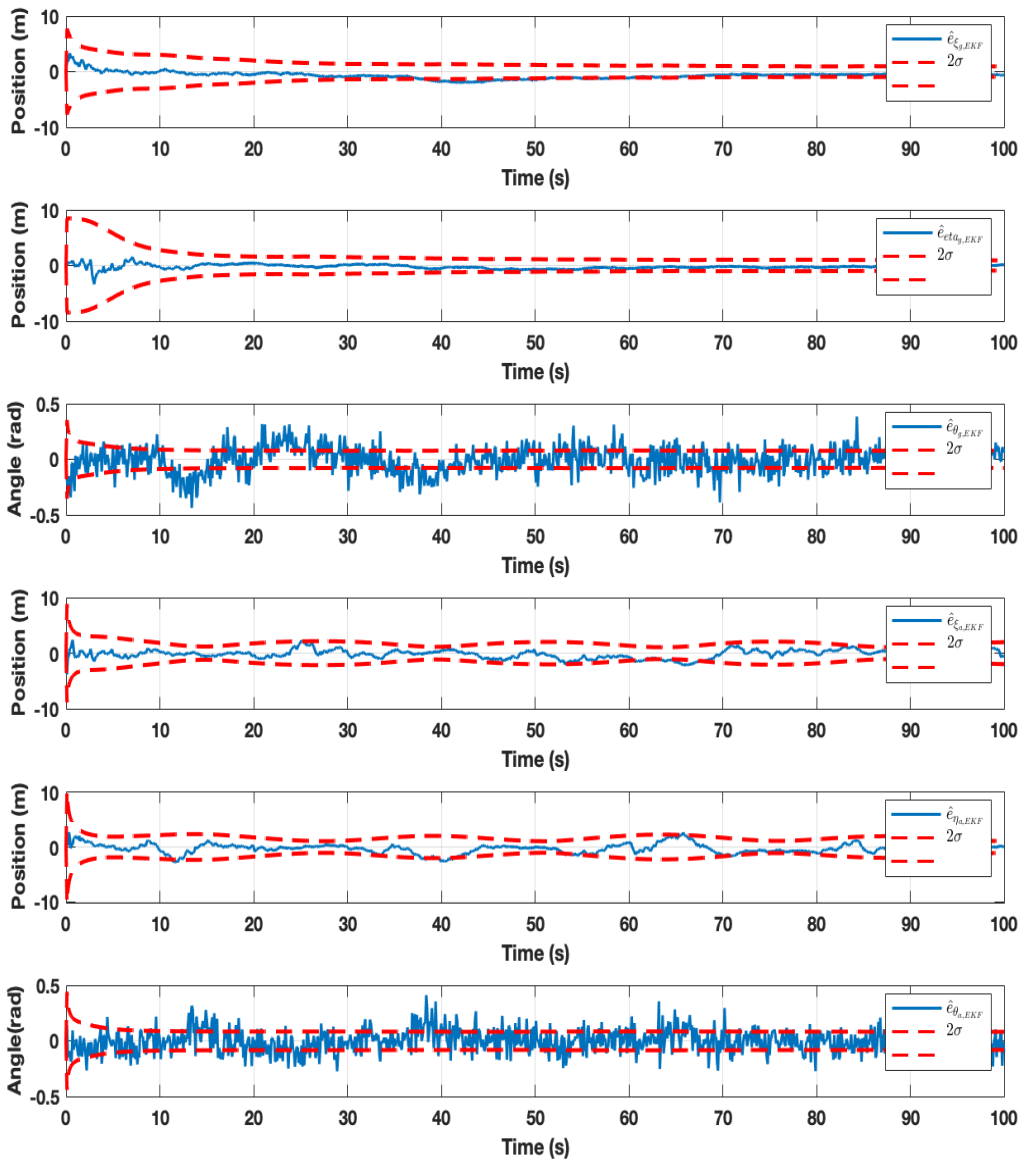

Figure 9: Initial EKF 2σ bounds and filter errors for all six states over 100 seconds.

From the above plot of EKF estimator errors, we can see a noticeable reduction in error and error excursion magnitude outside the 2σ bounds. As well, when the filter errors do extend past the bounds, the filter is able to recover much better than seen in the initial LKF impelentation results. This is due to the EKF utilizing a live trajectory update and total state estimate, instead of just state perturbation with respect to a fixed nominal trajectory. We can also observe this improvement in performance reflected in the plots of estimator state tracking and measurement innovation residuals.

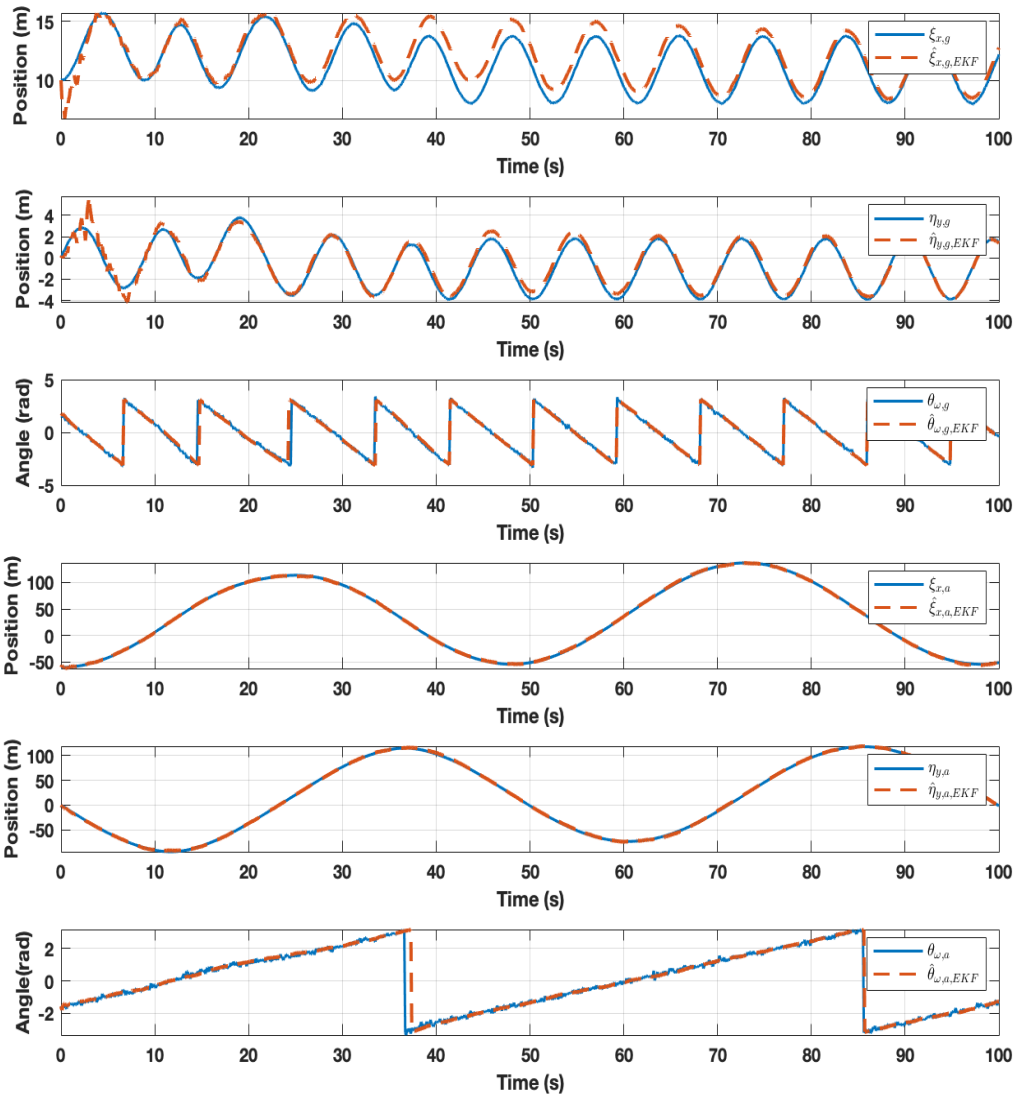

Figure 10: Initial EKF estimate tracking compared to ground truth data.

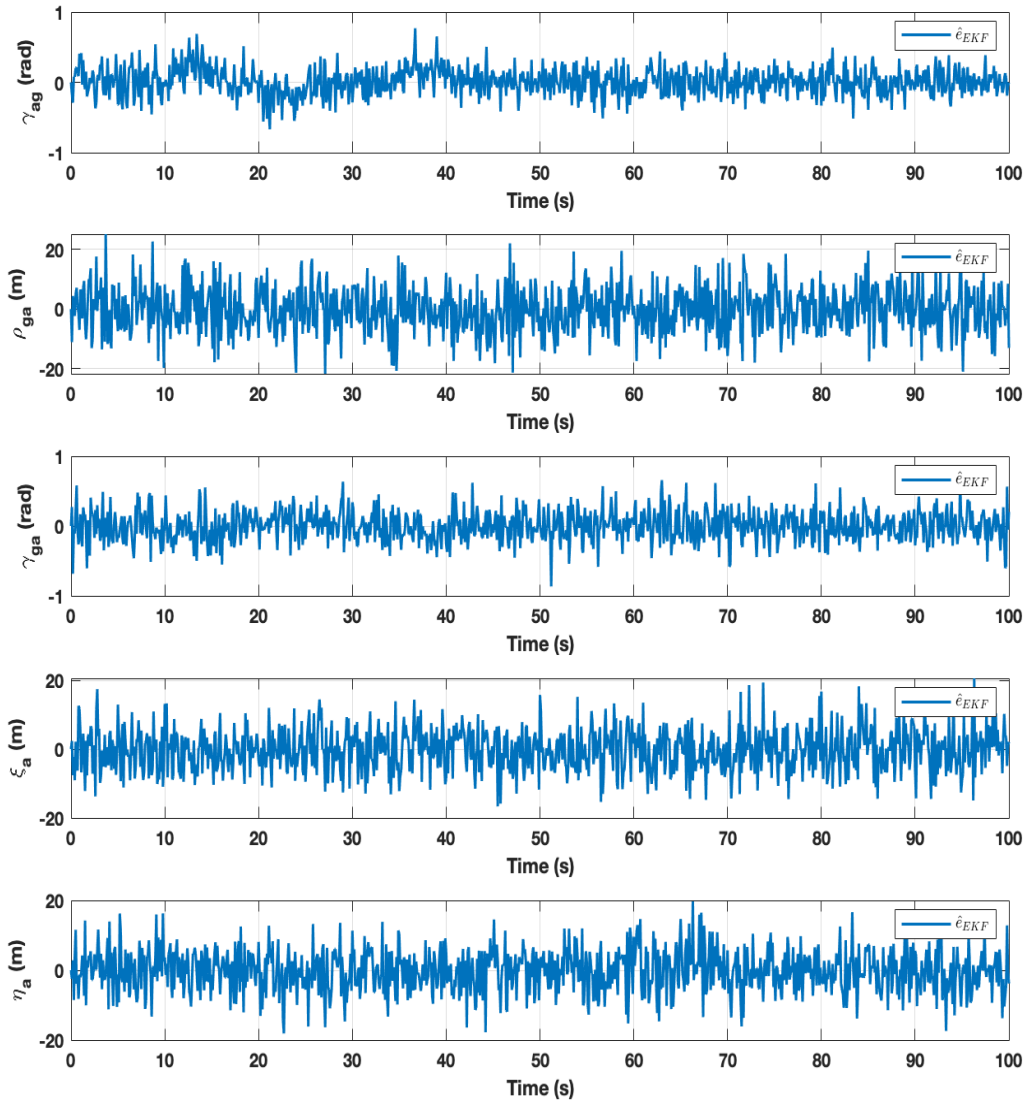

Figure 11: Initial EKF measurement innovation residuals for each state.

As expected from the estimator error results, the EKF estimator state tracking is significantly improved in comparison to the LKF. Additionally, the EKF measurement innovation residuals look like additive, zero-mean, white noise - which tells us that the filter is performing correctly. The implemented EKF was also tuned in the same manner as the LKF and simulated again after tuning. The simulation results for the tuned EKF implementation are shown below in Figure 12 and Figure 13.

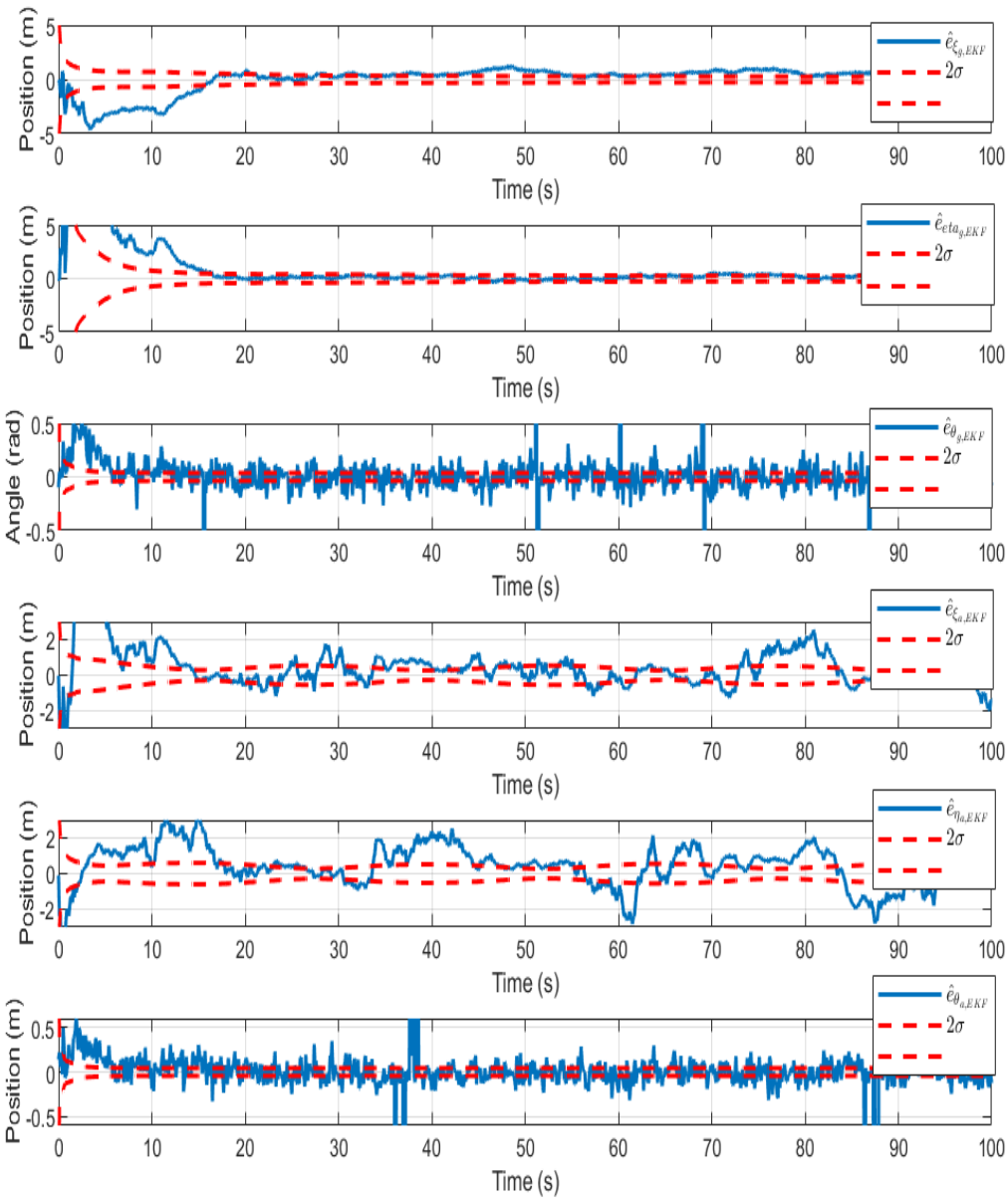

Figure 12: Tuned EKF 2σ bounds and filter errors for all six states over 100 seconds.

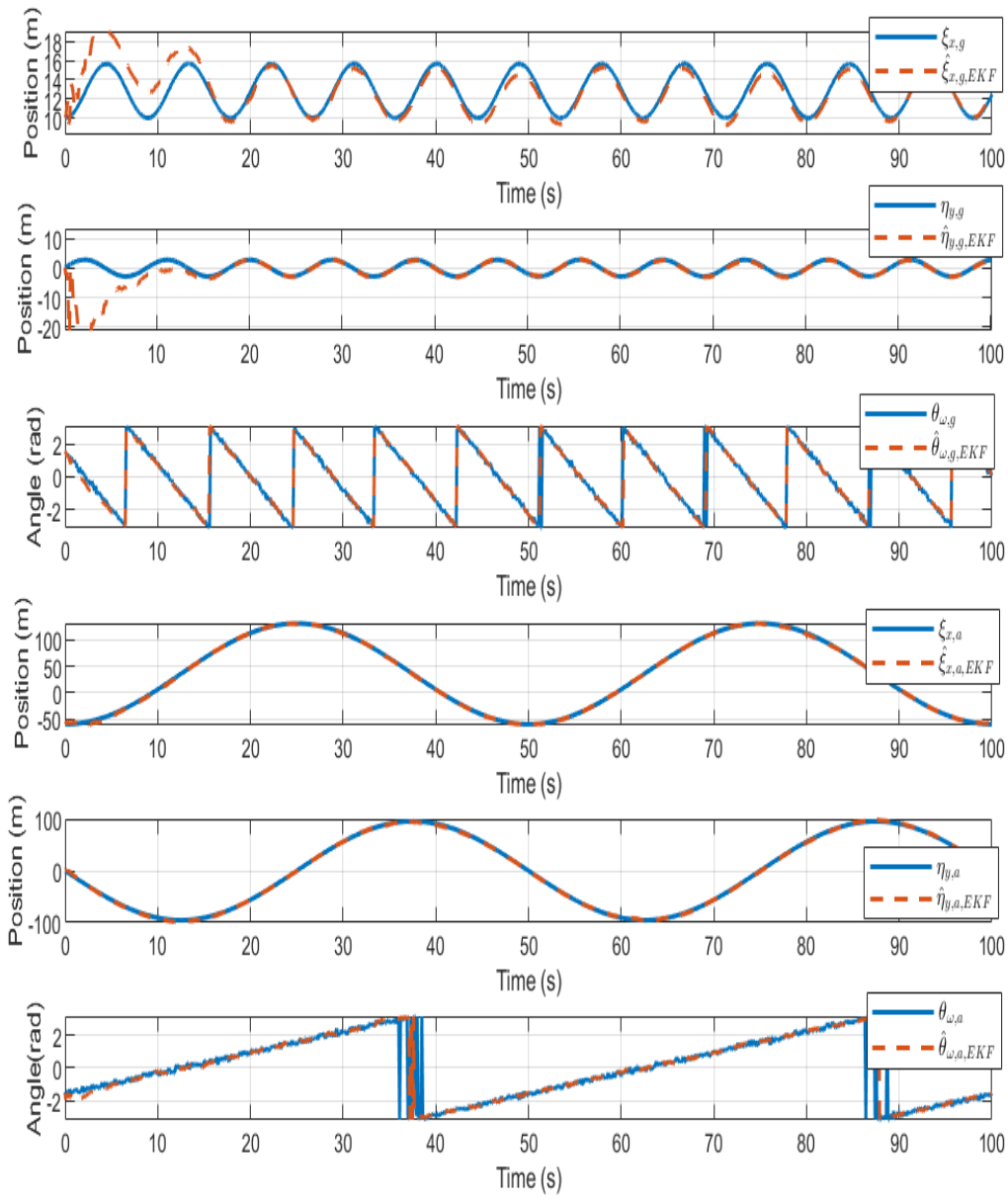

Figure 13: Tuned EKF estimate tracking compared to ground truth data.

After tuning the filter the simulation results show slightly better estimator tracking performance - as should be expected. Now that both Kalman Filters have been successfully implemented, the final section of the report will discuss the tuning process and test results.

6. Kalman Filter Truth Model Testing (TMT)

In order to test the two filters to assess their performance, we need to analyze their dynamic consistencies in state estimation error and measurement residuals. This is done by checking the normalized magnitude of these error vectors. The normalized estimation error squared (NEES) statistic and normalized innovation squared statistic (NIS) are computed by:

$$\boldsymbol{\epsilon}_{x,k} = \boldsymbol{e}_{x,k}^T (\boldsymbol{P}^+)^{-1} \boldsymbol{e}_{x,k} \tag{53}$$

$$\boldsymbol{\epsilon}_{y,k} = \boldsymbol{e}_{y,k}^T (\boldsymbol{S}_k)^{-1} \boldsymbol{e}_{y,k} \tag{54}$$

For the LKF, the state estimation error vector and measurement error vector are:

$$\boldsymbol{e}_{x,k} = \boldsymbol{x}_k - (\boldsymbol{x}_{k,nom} + \boldsymbol{\delta}\hat{\boldsymbol{x}}_k^+) \tag{55}$$

$$\boldsymbol{e}_{y,k} = \boldsymbol{y}_k - (\boldsymbol{y}_{k,nom} + \boldsymbol{\delta}\boldsymbol{y}_k) \tag{56}$$

For the EKF, the state estimation error vector and measurement error vector are:

$$\boldsymbol{e}_{x,k} = \boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^+ \tag{57}$$

$$\boldsymbol{e}_{y,k} = \boldsymbol{y}_k - \hat{\boldsymbol{y}}_k^- \tag{58}$$

The criteria for the NEES test is that the computed statistics should appear as a χ^2 distribution, given the number of degrees of freedom (n), for all time steps k. As well $\boldsymbol{E}[\boldsymbol{\epsilon}_{x,k}] = n$, and $var(\boldsymbol{\epsilon}_{x,k}) = 2n$. For the NIS test, the computed statistics should appear as a χ^2 distribution, given the number of observations (p) and $\boldsymbol{E}[\boldsymbol{\epsilon}_{y,k}] = p$ and $var(\boldsymbol{\epsilon}_{y,k}) = 2p$.

NIS and NEES statistics testing was performed with a significance level of .05. Filter tuning was an iterative process, where the filter's process noise and measurement error covariance matrices ($\boldsymbol{Q}_{LKF}$, $\boldsymbol{R}_{LKF}$) were altered to improve NEES and NIS statistics scores. For the testing, 50 Monte Carlo simulations were computed for the LKF and 25 for the EKF to generate the NEES and NIS statistics for each filter. The nominal values for $\boldsymbol{\mu}_0$ were used to initialize the mean for each filter. The LKF was initialized with $\hat{\boldsymbol{P}}_0^+ = 9 \cdot diag([2,\ 2,\ 2,\ 10,\ 10,\ 2])$. Figure 15 and Figure 14 show the results from the tuning tests using the following covariance matrices:

$$\boldsymbol{Q}_{LKF} = \begin{bmatrix} 0.0007 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.0007 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3.5625 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.0071 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.0071 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3.5625 \end{bmatrix} \tag{59}$$

$$\boldsymbol{R}_{LKF} = \begin{bmatrix} 0.0007 & 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0.0031 & 0 & 0 \\ 0 & 0 & 0 & 2.25 & 0 \\ 0 & 0 & 0 & 0 & 2.25 \end{bmatrix} \tag{60}$$

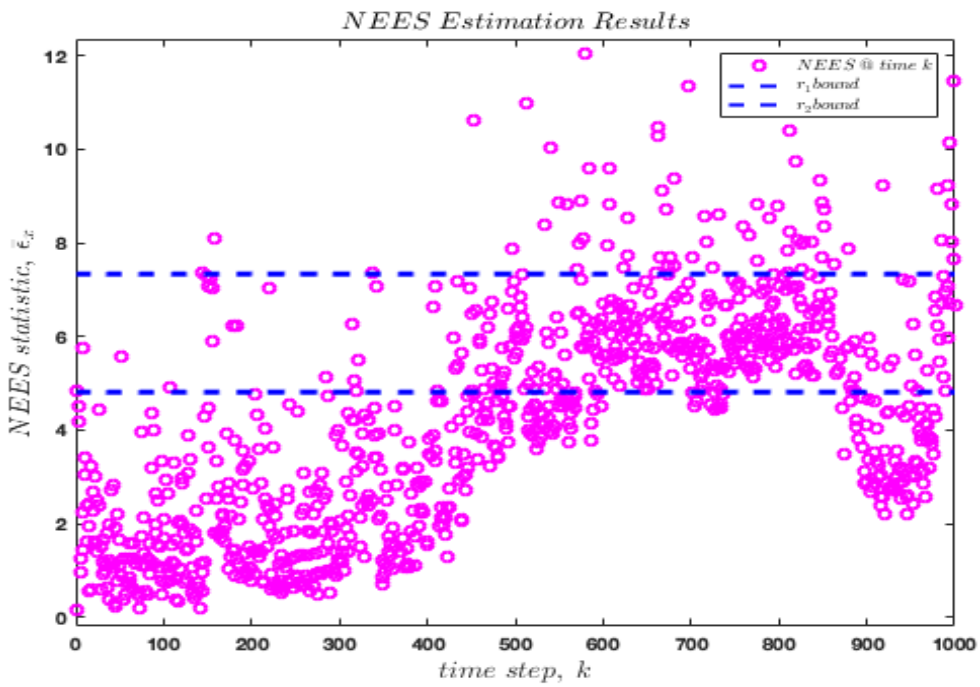

Figure 14: LKF NEES statistics testing results for all time steps. $r_{1,x}$ and $r_{2,x}$ bounds are 4.813 and 7.337, respectively

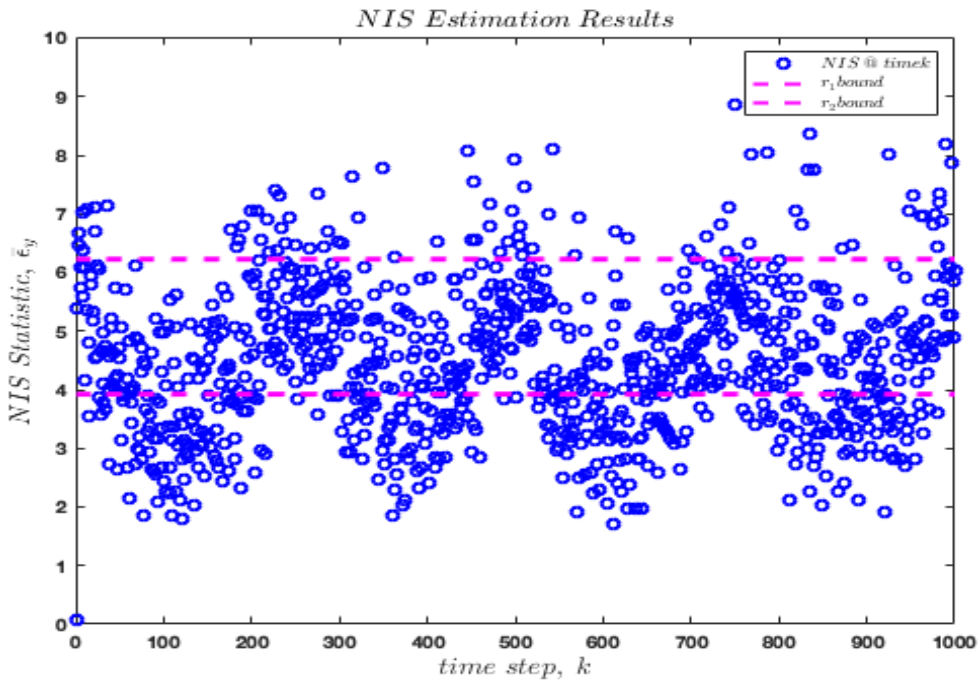

Figure 15: LKF NIS statistics testing results for all time steps. $r_{1,y}$ and $r_{2,y}$ bounds are 3.923 and 6.227, respectively.

These test results show fairly good performance with respect to the NIS criteria. However, performance does not fare as well with respect to the NEES criteria - only meeting criteria between time step 600 and time step 875. Again, we will see improved performance testing results from the EKF Monte-Carlo analysis. The EKF was initialized $\hat{\boldsymbol{P}}_0^+ = diag([64,\ 64,\ 4,\ 170,\ 170,\ 4])$. Figure 16 and Figure 17 show the results from the tuning tests using the following covariance matrices:

$$\boldsymbol{Q}_{EKF} = \begin{bmatrix} 0.04 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.04 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.04 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.04 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.9 \end{bmatrix} * \boldsymbol{Q}_{True} \tag{61}$$

$$\boldsymbol{R}_{EKF} = 0.06 * \boldsymbol{R}_{True} \tag{62}$$

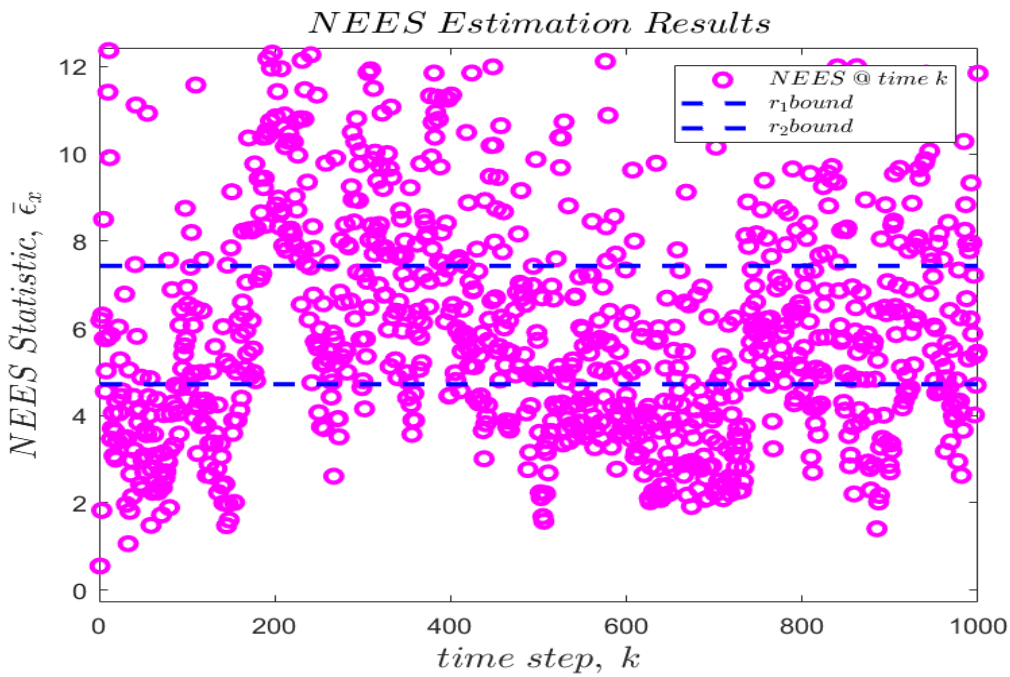

Figure 16: EKF NEES statistics testing results for all time steps. $r_{1,x}$ and $r_{2,x}$ bounds are 4.719 and 7.432, respectively

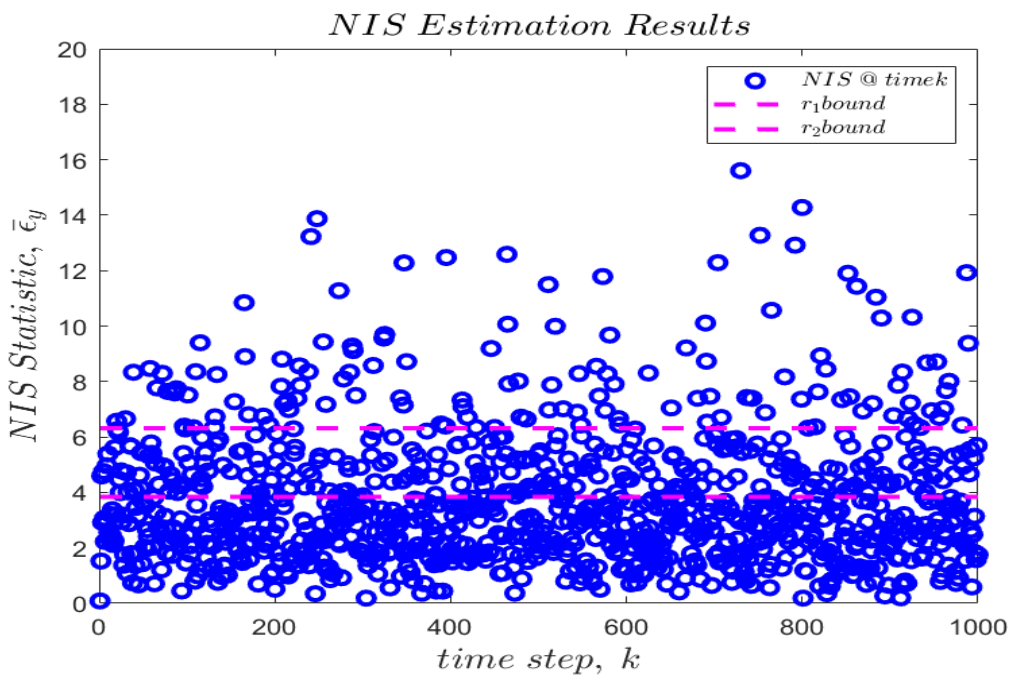

Figure 17: EKF NIS statistics testing results for all time steps. $r_{1,y}$ and $r_{2,y}$ bounds are 3.838 and 6.314, respectively.

These test results show a significant improvement from the LKF performance tests - with excellent results for the NIS criteria and decent performance for the The NEES criteria. As mention previously, we expected the EKF to outperform the LKF because of the innate limitations it has in handling large state or control input perturbations.

7. Tuned LKF & EKF Tuned Filter Implementation Results

Following the TMT runs, both filters were implemented using the sensor observation data provided for this project. The estimated states and 2σ values output from the Linearized Kalman Filter are provided below in Figure 18 and Figure 19.

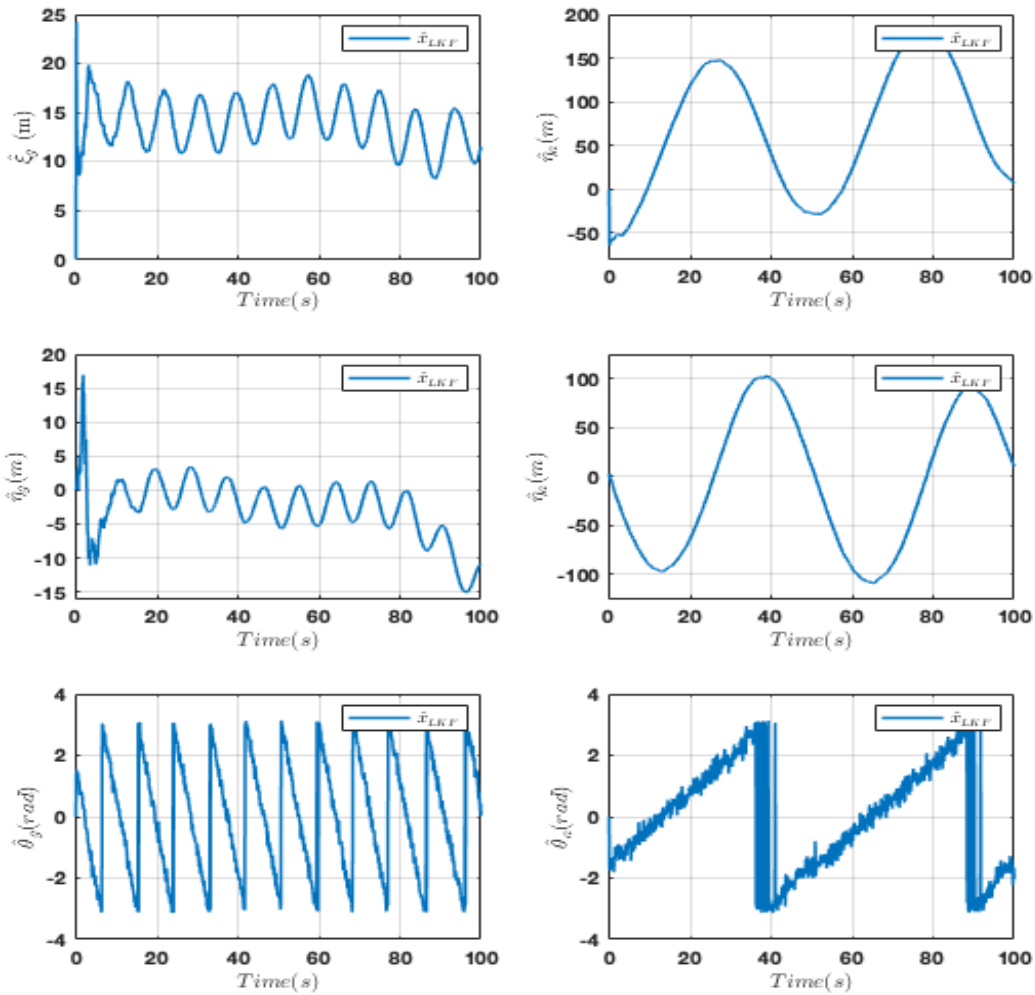

Figure 18: LKF state estimates from provided observation data.

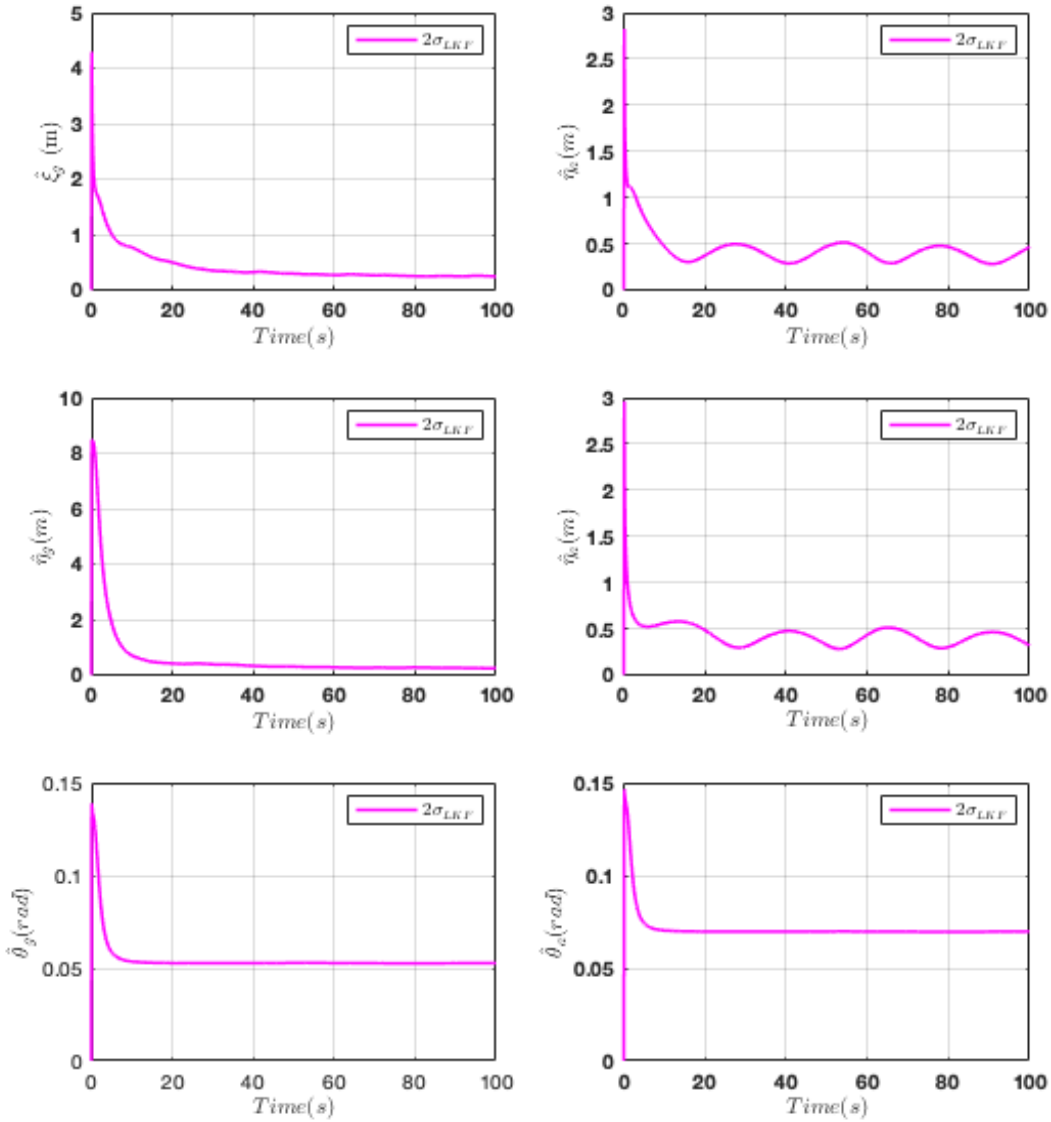

Figure 19: LKF state estimates from provided observation data.

The state estimate results from the LKF appear fairly noisy and large trajectory excursions are easily observable in states $\hat{\boldsymbol{x}}_1$ and $\hat{\boldsymbol{x}}_2$ at later time steps. Also, from the TMT results as well as knowing that the filter cannot manage larger perturbations well, we did not expect this filter to perform extremely well. Conversely, we do expect the Extended Kalman Filter to perform much better with the real data because the filter predicts the full state at every time step - instead of perturbations from an a priori trajectory. The state

estimate results and 2σ values output from the EKF are displayed below in Figure 20 and Figure 21.

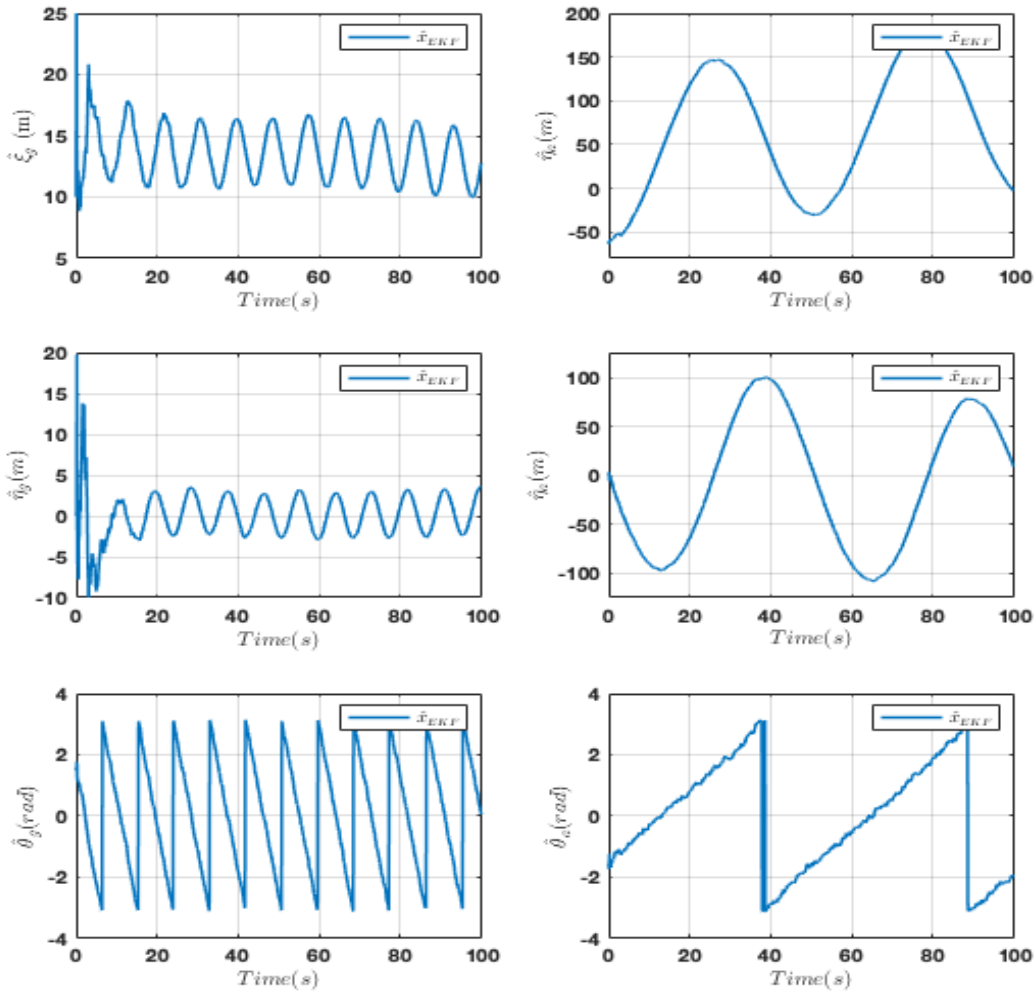

Figure 20: EKF state estimates from provided observation data.

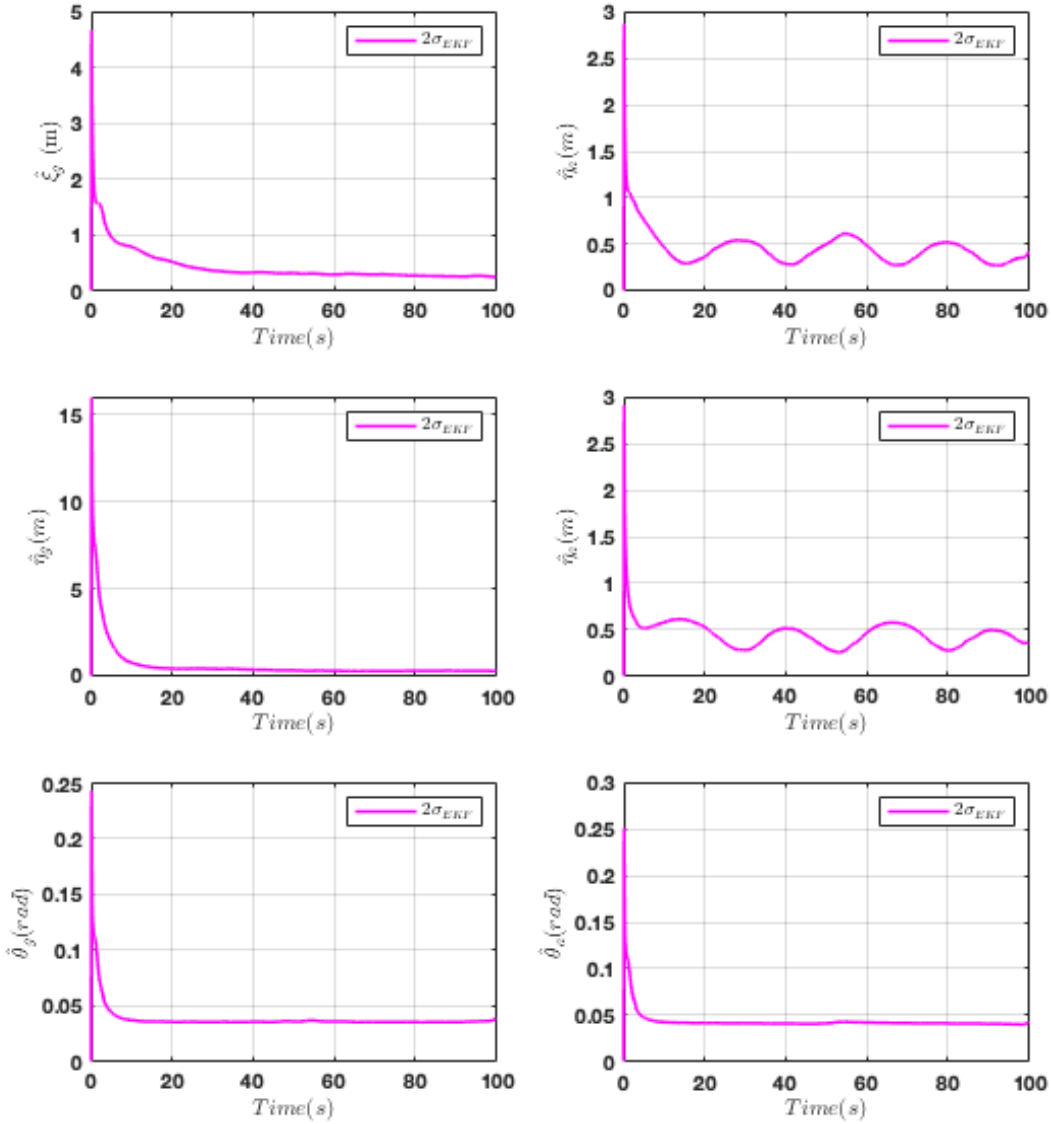

Figure 21: EKF state estimates from provided observation data.

While some state estimates appear noisy at very early time steps, we do see a significant improvement to the state estimates and moderate improvement in 2σ values in the EKF outputs. As well, the estimate trajectories do not fall off as much as we saw from the LKF outputs.

Appendix A. MATLAB Algorithm

```
%% ASEN 5044 Final Project

% Kemper Mahanes
% Henry Dixon
% CU Boulder
% Spring 2020

% UGV - UAV Robot Cooperative Localization

clc; clear; close all;

%% Cooperative System Parameters & Data

%----------------------%
% Nominal Parameters %
%----------------------%

% UGV
L = 0.5; %(m) UGV wheel base length
phig_rng = [-(5*pi)/12,(5*pi)/12]; %(rad) UGV steering angle range
vg_max = 3; %(m/s) max speed
xg_0 = [10; 0; pi/2]; % initial values for UGV states
ug_0 = [2; -pi/18]; % initial values for UGV input states
ug_0L = [2; -5*pi/12]; % lower bound for nominal control values
ug_0U = [3; 5*pi/12]; % upper bound for nominal control values

% UAV
wg_rng = [-pi/6, pi/6]; %(rad/s) turn rate range
va_rng = [10 20]; %(m/s) velocity range
xa_0 = [-60; 0; -pi/2]; % initial values for aircraft states
ua_0 = [12; pi/25]; % initial control input states for aircraft
ua_0L = [12; -pi/6]; % lower bound for nominal control values
ua_0U = [12; pi/6]; % upper bound for nominal control values

% Combined Initial State & Control Values
x0 = [xg_0; xa_0];
u0 = [ug_0; ua_0];
u0_U = [ug_0U;ua_0U];
u0_L = [ug_0L;ua_0L];
mu_x = x0;

% DT Time Step
dt = .1; %(s) time step

% Load Data for Implementing KF and EKF
load('cooplocalization_finalproj_KFdata.mat')

% Measurement & Process Noise Covar Matrices
Q = Qtrue;
R = Rtrue;

% Gamma and Omega Matrices
Gamma = eye(6);
```

```
Omega = dt*Gamma;
Svx = chol(Q);
Svy = chol(R);
rng(100);

%% Model Simulation & Computation of Nominal Trajectory/Truth Data

tspan = 0:dt:100; %(s) time span
T = length(tspan);
n = length(x0); % number of states

% Pertubation for forcing function vector
u = @(t) u0.*[(.2*sin(10*pi*t)),.25,-.5*cos(10*pi*t),.5]' * ((10<t) & (t<14))
    ...
    + u0.*[(-.2*sin(20*pi*t)),-.25,.5*cos(10*pi*t),-.5]' * ((20<t) & (t<24))
    ...
    + u0.*[(.2*sin(30*pi*t)),.25,-.5*cos(10*pi*t),.5]' * ((35<t) & (t<39));

% State Perturbation Vector & Nonlinear Model Simulation (for comparison to
    reference plots)
perturb_x0 = [0; 1; 0; 0; 0; 0.1];
x0p = x0 + perturb_x0;
dx = zeros(1,6);
[t,x_nl_check] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan,x0p);
x_nl_check(:,3) = wrapToPi(x_nl_check(:,3));
x_nl_check(:,6) = wrapToPi(x_nl_check(:,6));
x_nl_check = x_nl_check'; % transpose states to row vectors

% Nonlinear Model Response
[t,x_nl] = ode45(@(t,x) x_nl_traject(x,L,u,u0,t),tspan,x0);
x_nl(:,3) = wrapToPi(x_nl(:,3));
x_nl(:,6) = wrapToPi(x_nl(:,6));
x_nl = x_nl';

% Generate Ground Truth Data
x_truth = x_nl + Svx'*randn(n,T);
x_truth_check = x_nl_check + Svx'*randn(n,T);
ydata = nl_sensor_model_noise(x_truth,R);

% Generate Offline Nominal Trajectory from Nonlinear Model Dynamics
[t,xnom] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan,x0);
xnom(:,3) = wrapToPi(xnom(:,3));
xnom(:,6) = wrapToPi(xnom(:,6));
xnom = xnom';

% Compute Observation Data from Nonlinear Sensor Model
y_obs = nl_sensor_model(x_nl_check);
ynom = nl_sensor_model(xnom);


%% Linearized DT Model Simulation

% Initial Values for Simulation
x = x0; dx = perturb_x0; ylin = [];

```

```
% Simulation Loop
for k = 2:length(t)
%     U(:,k) = u(t(k)) + u0;

    % Update U and Linearized Matrices
    U(:,k) = u0;
    [~,~,F,G,H] = matrixUpdate(xnom(:,k-1),u0,dt,L);

    % Update States
    du(:,k-1) = U(:,k-1) - u0;
    dx(:,k) = F*dx(:,k-1) + G*du(:,k-1);
    x(:,k) = xnom(:,k) + dx(:,k);
    dy = H*dx(:,k);
    ylin(:,k) = ynom(:,k)+dy;
end

% Wrap Sensed Angles to [-pi pi]
ylin(1,:) = wrapToPi(ylin(1,:));
ylin(3,:) = wrapToPi(ylin(3,:));

%%
%-----------------------------------------------------------------------%
% Plot Nominal Nonlinear Trajectories vs Perturbed Trajectories %
%-----------------------------------------------------------------------%
f = 1;
figure(f)
f = f+1;
subplot(611)
plot(t,x_nl_check(1,:),'LineWidth',1.5)
grid on; hold on;
plot(t,xnom(1,:),'--','LineWidth',1.5)
ylim([8 18]);
xlabel('Time (s)'); ylabel('Position (m)');
legend('\xi_{x,g}','\xi_{x,g,nom}')

subplot(612)
plot(t,x_nl_check(2,:),'LineWidth',1.5)
grid on; hold on;
plot(t,xnom(2,:),'--','LineWidth',1.5)
ylim([-6 6]);
xlabel('Time (s)'); ylabel('Position (m)');
legend('\eta_{y,g}','\eta_{y,g,nom}')

subplot(613)
plot(t,x_nl_check(3,:),'LineWidth',1.5)
grid on; hold on;
plot(t,xnom(3,:),'--','LineWidth',1.5)
ylim([-4 4]);
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('\theta_{\omega,g}','\theta_{\omega,g,nom}')

subplot(614)
plot(t,x_nl_check(4,:),'LineWidth',1.5)
grid on; hold on;
plot(t,xnom(4,:),'--','LineWidth',1.5)
```

```
ylim([-75 150]);
xlabel('Time (s)'); ylabel('Position (m)');
legend('\xi_{x,a}','\xi_{x,a,nom}')

subplot(615)
plot(t,x_nl_check(5,:),'LineWidth',1.5)
grid on; hold on;
plot(t,xnom(5,:),'--','LineWidth',1.5)
ylim([-125 125]);
xlabel('Time (s)'); ylabel('Position (m)');
legend('\eta_{y,a}','\eta_{y,a,nom}')

subplot(616)
plot(t,x_nl_check(6,:),'LineWidth',1.5)
grid on; hold on;
plot(t,xnom(6,:),'--','LineWidth',1.5)
ylim([-4 4]);
xlabel('Time (s)'); ylabel('Angle(rad)');
legend('\theta_{\omega,a}','\theta_{\omega,a,nom}')

%%
%-----------------------------------------------------------------%
% Plot DT Linearized Model vs Actual Nonlinear Response %
%-----------------------------------------------------------------%

figure(f)
f = f+1;
subplot(611)
plot(t,x_nl_check(1,:),'LineWidth',1.5)
grid on; hold on;
plot(t,x(1,:),'--','LineWidth',1.5)
ylim([8 18]);
xlabel('Time (s)'); ylabel('Position (m)');
legend('\xi_{x,g}','\xi_{x,g,Lin}')

subplot(612)
plot(t,x_nl_check(2,:),'LineWidth',1.5)
grid on; hold on;
plot(t,x(2,:),'--','LineWidth',1.5)
ylim([-6 6]);
xlabel('Time (s)'); ylabel('Position (m)');
legend('\eta_{y,g}','\eta_{y,g,Lin}')

subplot(613)
plot(t,x_nl_check(3,:),'LineWidth',1.5)
grid on; hold on;
plot(t,x(3,:),'--','LineWidth',1.5)
ylim([-4 4]);
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('\theta_{\omega,g}','\theta_{\omega,g,Lin}')

subplot(614)
plot(t,x_nl_check(4,:),'LineWidth',1.5)
grid on; hold on;
plot(t,x(4,:),'--','LineWidth',1.5)
```

```
ylim([-75 150]);
xlabel('Time (s)'); ylabel('Position (m)');
legend('\xi_{x,a}','\xi_{x,a,Lin}')

subplot(615)
plot(t,x_nl_check(5,:),'LineWidth',1.5)
grid on; hold on;
plot(t,x(5,:),'--','LineWidth',1.5)
ylim([-125 125]);
xlabel('Time (s)'); ylabel('Position (m)');
legend('\eta_{y,a}','\eta_{y,a,Lin}')

subplot(616)
plot(t,x_nl_check(6,:),'LineWidth',1.5)
grid on; hold on;
plot(t,x(6,:),'--','LineWidth',1.5)
ylim([-4 4]);
xlabel('Time (s)'); ylabel('Angle(rad)');
legend('\theta_{\omega,a}','\theta_{\omega,a,Lin}')

%%
%-----------------------------------------------------------------------%
% Plot Linearized Sensor Model Outputs vs Actual Nonlinear Response %
%-----------------------------------------------------------------------%

% Plot Simulated Sensor Outputs
figure(f)
f = f+1;
subplot(511)
plot(t,y_obs(1,:),'LineWidth',1.5)
grid on; hold on;
plot(t,ylin(1,:),'--','LineWidth',1.5)
ylim([-4 4]);
xlabel('Time (s)'); ylabel('\gamma_{ag} (rad)');
legend('Nonlinear','Linear')

subplot(512)
plot(t,y_obs(2,:),'LineWidth',1.5)
grid on; hold on;
ylim([40 130]);
plot(t,ylin(2,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('\rho_{ga} (m)');
legend('Nonlinear','Linear')

subplot(513)
plot(t,y_obs(3,:),'LineWidth',1.5)
grid on; hold on;
plot(t,ylin(3,:),'--','LineWidth',1.5)
ylim([1 2.1]);
xlabel('Time (s)'); ylabel('\gamma_{ga} (rad)');
legend('Nonlinear','Linear')

subplot(514)
plot(t,y_obs(4,:),'LineWidth',1.5)
grid on; hold on;
```

```
plot(t,ylin(4,:),'--','LineWidth',1.5)
ylim([-75 150]);
xlabel('Time (s)'); ylabel('\xi_a (m)');
legend('Nonlinear','Linear')

subplot(515)
plot(t,y_obs(5,:),'LineWidth',1.5)
grid on; hold on;
plot(t,ylin(5,:),'--','LineWidth',1.5)
ylim([-125 125]);
xlabel('Time (s)'); ylabel('\eta_a (m)');
legend('Nonlinear','Linear')

%-------------------------------------------------------------%
%            END OF CODE FOR PART 1 OF PROJECT                %
%-------------------------------------------------------------%

%% Kalman Filter Implementation

%-------------------------------%
% Run Linearized Kalman Filter %
%-------------------------------%

% Nominal Forcing Function
ustar = ones(1,T).*u0;

% Initialize LKF
% tune_factQ = 950*0.075;
% Qlkf = diag([.01 .01 5 .1 .1 5])*Qtrue*tune_factQ;
% Rlkf = diag([.5 1 .5 1 1])*Rtrue*.0625;


% Qkf=diag(1000*[.01 .01 1 .05 .05 1])*Q; %%treat this as the main tuning knob
% Rkf = .075*R; %%assume we can get this from sensor specs
sf = 0.075;
Qlkf=diag(950*[.01 .01 5 .1 .1 5])*Q; %%treat this as the main tuning knob
Rlkf = diag([.5 1 .5 1 1])*R*.0625; %%assume we can get this from sensor specs
Qlkf = Qlkf*sf;

Po = 9*diag([2 2 2 10 10 2]);
Pkp = reshape(Po,6^2,1);
dxhat_p = zeros(n,1);

% Linearized Kalman Filter Loop
for k = 1:length(ydata)-1

    % Prediction Step
    [~,~,F,G,H] = matrixUpdate(xnom(:,k),u0,dt,L);
    du = u0 - ustar(:,k);
    dxhat_m = F*dxhat_p(:,k) + G*du;
    Pkm = F*reshape(Pkp(:,k),6,6)*F' + Omega*Qlkf*Omega';

    % Measurement Update Step
    [~,~,~,~,H] = matrixUpdate(xnom(:,k+1),u0,dt,L);
    dy(:,k+1) = ydata(:,k+1) - ynom(:,k+1);
```

```
    dy(1,k+1) = wrapToPi(dy(1,k+1)); dy(3,k+1) = wrapToPi(dy(3,k+1));
    K = Pkm*H'*inv(H*Pkm*H' + Rlkf);
    Pp = (eye(6) - K*H)*Pkm;
    Pkp(:,k+1) = reshape(Pp,6^2,1);

    % Update Total State Estimates
    dxhat_p(:,k+1) = dxhat_m + K*(dy(:,k+1) - H*dxhat_m);
    dxhat_p(3,k+1) = dxhat_p(3,k+1); dxhat_p(6,k+1) = dxhat_p(6,k+1);
    xhat(:,k+1) = xnom(:,k+1) + dxhat_p(:,k+1);
    xhat(3,k+1) = wrapToPi(xhat(3,k+1)); xhat(6,k+1) = wrapToPi(xhat(6,k+1));
    yhat(:,k+1) = ynom(:,k+1) + dy(:,k+1);
    yhat(1,k+1) = wrapToPi(yhat(1,k+1)); yhat(3,k+1) = wrapToPi(yhat(3,k+1));

    % Estimator Error & Sigma Values
    sig_lkf(:,k+1) = sqrt(diag(Pp));
    e_xhat(:,k+1) = x_truth(:,k+1) - xhat(:,k+1);
    e_xhat(3,k+1) = wrapToPi(e_xhat(3,k+1));
    e_xhat(6,k+1) = wrapToPi(e_xhat(6,k+1));

    e_yhat(:,k+1) = ydata(:,k+1) - yhat(:,k+1);
    e_yhat(1,k+1) = wrapToPi(e_yhat(1,k+1));
    e_yhat(3,k+1) = wrapToPi(e_yhat(3,k+1));
end


%%
%-----------------------------------------%
% Plot LKF Estimates vs  Ground Truth %
%-----------------------------------------%

figure(f)
f = f+1;
subplot(611)
plot(tvec,x_truth(1,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat(1,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\xi_{x,g}$','$\hat{\xi}_{x,g,LKF}$','Interpreter','latex');

subplot(612)
plot(tvec,x_truth(2,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat(2,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\eta_{y,g}$','$\hat{\eta}_{y,g,LKF}$','Interpreter','latex');

subplot(613)
plot(tvec,x_truth(3,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat(3,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\theta_{\omega,g}$','$\hat{\theta}_{\omega,g,LKF}$','Interpreter','
    latex');

subplot(614)
```

```
plot(tvec,x_truth(4,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat(4,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\xi_{x,a}$','$\hat{\xi}_{x,a,LKF}$','Interpreter','latex');

subplot(615)
plot(tvec,x_truth(5,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat(5,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\eta_{y,a}$','$\hat{\eta}_{y,a,LKF}$','Interpreter','latex');

subplot(616)
plot(tvec,x_truth(6,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat(6,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle(rad)');
legend('$\theta_{\omega,a}$','$\hat{\theta}_{\omega,a,LKF}$','Interpreter','
    latex');

%%
%-----------------------------------------------------%
% Plot LKF State Estimate Error & 2 Sigma Bounds %
%-----------------------------------------------------%

figure(f)
f = f+1;
subplot(611)
plot(tvec,e_xhat(1,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_lkf(1,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_lkf(1,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{\xi_{g}}$','$2\sigma$','','Interpreter','latex');

subplot(612)
plot(tvec,e_xhat(2,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_lkf(2,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_lkf(2,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{eta_{g}}$','$2\sigma$','','Interpreter','latex');

subplot(613)
plot(tvec,e_xhat(3,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_lkf(3,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_lkf(3,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\hat{e}_{\theta_{g}}$','$2\sigma$','','Interpreter','latex');

subplot(614)
plot(tvec,e_xhat(4,:),'LineWidth',1.5)
grid on; hold on;
```

```
plot(tvec,2*sig_lkf(4,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_lkf(4,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{\xi_{a}}$','$2\sigma$','','Interpreter','latex');

subplot(615)
plot(tvec,e_xhat(5,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_lkf(5,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_lkf(5,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{\eta_{a}}$','$2\sigma$','','Interpreter','latex');

subplot(616)
plot(tvec,e_xhat(6,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_lkf(6,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_lkf(6,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\hat{e}_{\theta_{a}}$','$2\sigma$','','Interpreter','latex');


%%
%-------------------------------------------------------------------------%
% Plot Linearized Sensor Model Outputs vs Actual Nonlinear Response %
%-------------------------------------------------------------------------%

% Plot Simulated Sensor Outputs
figure(f)
f = f+1;
subplot(511)
plot(tvec,ydata(1,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,yhat(1,:),'--','LineWidth',1.5)
% ylim([-4 4]);
xlabel('Time (s)'); ylabel('\gamma_{ag} (rad)');
legend('$Data$','$\hat{y}_{LKF}$','Interpreter','latex');

subplot(512)
plot(tvec,ydata(2,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,yhat(2,:),'--','LineWidth',1.5)
% ylim([40 130]);
xlabel('Time (s)'); ylabel('\rho_{ga} (m)');
legend('$Data$','$\hat{y}_{LKF}$','Interpreter','latex');

subplot(513)
plot(tvec,ydata(3,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,yhat(3,:),'--','LineWidth',1.5)
% ylim([1 2.1]);
xlabel('Time (s)'); ylabel('\gamma_{ga} (rad)');
legend('$Data$','$\hat{y}_{LKF}$','Interpreter','latex');

subplot(514)
```

```
plot(tvec,ydata(4,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,yhat(4,:),'--','LineWidth',1.5)
% ylim([-75 150]);
xlabel('Time (s)'); ylabel('\xi_a (m)');
legend('$Data$','$\hat{y}_{LKF}$','Interpreter','latex');

subplot(515)
plot(tvec,ydata(5,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,yhat(5,:),'--','LineWidth',1.5)
% ylim([-125 125]);
xlabel('Time (s)'); ylabel('\eta_a (m)');
legend('$Data$','$\hat{y}_{LKF}$','Interpreter','latex');

%-------------------------------------------------------%
% Plot LKF Measurement Estimator Error (Innovation)  %
%-------------------------------------------------------%

% Plot Simulated Sensor Outputs
figure(f)
f = f+1;
subplot(511)
plot(tvec,dy(1,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\delta y_{\gamma_{a,g}}$','Interpreter','latex');

subplot(512)
plot(tvec,dy(2,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\delta y_{\rho_{ga}}$','Interpreter','latex');

subplot(513)
plot(tvec,dy(3,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\delta y_{\gamma_{g,a}}$','Interpreter','latex');

subplot(514)
plot(tvec,dy(4,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\delta y_{\xi_a}$','Interpreter','latex');

subplot(515)
plot(tvec,dy(5,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\delta y_{\eta_a}$','Interpreter','latex');

%%
%-----------------------------%
% Run Extended Kalman Filter %
```

```
%-------------------------------%

% Initialize EKF
Qekf = diag([.04 .04 .9 .04 .04 .9])*Qtrue;
Rekf = .06*Rtrue;
Po = diag([64, 64, 64, 170, 170 4]);
Pkp_ekf = reshape(Po,6^2,1);
tspan2 = 0:.1:.1;
xhat_ekf = mu_x;
dx = zeros(6,1);
% Extended Kalman Filter Loop
for k = 1:length(ydata)-1

    % Prediction Step
    [t,xnew] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan2,xhat_ekf(:,k)
    );
    xhat_m = xnew(end,:)';
    [~,~,F,~,~] = matrixUpdate(xhat_ekf(:,k),u0,dt,L);
    Pkm = F*reshape(Pkp_ekf(:,k),6,6)*F' + Omega*Qekf*Omega';

    % Correction Step
    [~,~,~,~,H] = matrixUpdate(xhat_m,u0,dt,L);
    yhat_m(:,k+1) = nl_sensor_model(xhat_m);
    ey(:,k+1) = ydata(:,k+1) - yhat_m(:,k+1);
    ey(1,k+1) = wrapToPi(ey(1,k+1));
    ey(3,k+1) = wrapToPi(ey(3,k+1));

    K = Pkm*H'*inv(H*Pkm*H' + Rekf);
    xhat_ekf(:,k+1) = xhat_m + K*ey(:,k+1);
    xhat_ekf(3,k+1) = wrapToPi(xhat_ekf(3,k+1));
    xhat_ekf(6,k+1) = wrapToPi(xhat_ekf(6,k+1));
    Pp = (eye(6) - K*H)*Pkm;
    Pkp_ekf(:,k+1) = reshape(Pp,6^2,1);

    % Estimator Error & Sigma Values
    sig_ekf(:,k+1) = sqrt(diag(Pp));
    e_xhat_ekf(:,k+1) = x_truth(:,k+1) - xhat_ekf(:,k+1);
    e_xhat_ekf(3,k+1) = wrapToPi(e_xhat_ekf(3,k+1));
    e_xhat_ekf(6,k+1) = wrapToPi(e_xhat_ekf(6,k+1));

end

%%
%-----------------------------------------%
% Plot EKF Estimates vs  Ground Truth %
%-----------------------------------------%

figure(f)
f = f+1;
subplot(611)
plot(tvec,x_truth(1,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat_ekf(1,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\xi_{x,g}$','$\hat{\xi}_{x,g,EKF}$','Interpreter','latex');
```

```
subplot(612)
plot(tvec,x_truth(2,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat_ekf(2,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\eta_{y,g}$','$\hat{\eta}_{y,g,EKF}$','Interpreter','latex');

subplot(613)
plot(tvec,x_truth(3,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat_ekf(3,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\theta_{\omega,g}$','$\hat{\theta}_{\omega,g,EKF}$','Interpreter','latex');

subplot(614)
plot(tvec,x_truth(4,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat_ekf(4,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\xi_{x,a}$','$\hat{\xi}_{x,a,EKF}$','Interpreter','latex');

subplot(615)
plot(tvec,x_truth(5,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat_ekf(5,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\eta_{y,a}$','$\hat{\eta}_{y,a,EKF}$','Interpreter','latex');

subplot(616)
plot(tvec,x_truth(6,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhat_ekf(6,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle(rad)');
legend('$\theta_{\omega,a}$','$\hat{\theta}_{\omega,a,EKF}$','Interpreter','latex');

%%
%-----------------------------------------------------%
% Plot EKF State Estimate Error & 2 Sigma Bounds %
%-----------------------------------------------------%

figure(f)
f = f+1;
subplot(611)
plot(tvec,e_xhat_ekf(1,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_ekf(1,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_ekf(1,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{\xi_{g,EKF}}$','$2\sigma$','','Interpreter','latex');

subplot(612)
plot(tvec,e_xhat_ekf(2,:),'LineWidth',1.5)
```

```
grid on; hold on;
plot(tvec,2*sig_ekf(2,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_ekf(2,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{eta_{g,EKF}}$','$2\sigma$','','Interpreter','latex');

subplot(613)
plot(tvec,e_xhat_ekf(3,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_ekf(3,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_ekf(3,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\hat{e}_{\theta_{g,EKF}}$','$2\sigma$','','Interpreter','latex');

subplot(614)
plot(tvec,e_xhat_ekf(4,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_ekf(4,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_ekf(4,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{\xi_{a,EKF}}$','$2\sigma$','','Interpreter','latex');

subplot(615)
plot(tvec,e_xhat_ekf(5,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_ekf(5,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_ekf(5,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{\eta_{a,EKF}}$','$2\sigma$','','Interpreter','latex');

subplot(616)
plot(tvec,e_xhat_ekf(6,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sig_ekf(6,:),'r--','LineWidth',1.5)
plot(tvec,-2*sig_ekf(6,:),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle(rad)');
legend('$\hat{e}_{\theta_{a,EKF}}$','$2\sigma$','','Interpreter','latex');


%%
%-----------------------------------%
% Plot Sensor Estimates vs YDATA %
%-----------------------------------%

%% Plot Simulated Sensor Outputs
% figure(f)
% f = f+1;
% subplot(511)
% plot(tvec,ydata(1,:),'LineWidth',1.5)
% grid on; hold on;
% plot(tvec,yhat_m(1,:),'--','LineWidth',1.5)
%% ylim([-4 4]);
% xlabel('Time (s)'); ylabel('\gamma_{ag} (rad)');
% legend('$Data$','$\hat{y}_{EKF}$','Interpreter','latex');
%
```

```
% subplot(512)
% plot(tvec,ydata(2,:),'LineWidth',1.5)
% grid on; hold on;
% plot(tvec,yhat_m(2,:),'--','LineWidth',1.5)
%% ylim([40 130]);
% xlabel('Time (s)'); ylabel('\rho_{ga} (m)');
% legend('$Data$','$\hat{y}_{EKF}$','Interpreter','latex');
%
% subplot(513)
% plot(tvec,ydata(3,:),'LineWidth',1.5)
% grid on; hold on;
% plot(tvec,yhat_m(3,:),'--','LineWidth',1.5)
%% ylim([1 2.1]);
% xlabel('Time (s)'); ylabel('\gamma_{ga} (rad)');
% legend('$Data$','$\hat{y}_{EKF}$','Interpreter','latex');
%
% subplot(514)
% plot(tvec,ydata(4,:),'LineWidth',1.5)
% grid on; hold on;
% plot(tvec,yhat_m(4,:),'--','LineWidth',1.5)
%% ylim([-75 150]);
% xlabel('Time (s)'); ylabel('\xi_a (m)');
% legend('$Data$','$\hat{y}_{EKF}$','Interpreter','latex');
%
% subplot(515)
% plot(tvec,ydata(5,:),'LineWidth',1.5)
% grid on; hold on;
% plot(tvec,yhat_m(5,:),'--','LineWidth',1.5)
%% ylim([-125 125]);
% xlabel('Time (s)'); ylabel('\eta_a (m)');
% legend('$Data$','$\hat{y}_{EKF}$','Interpreter','latex');
%
%%
%-----------------------------------------------------%
% Plot EKF Sensor Estimator Error (Innovation)  %
%-----------------------------------------------------%

% Plot Simulated Sensor Outputs
figure(f)
f = f+1;
subplot(511)
plot(tvec,ey(1,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\tilde{e}_{y,\gamma_{a,g}}$','Interpreter','latex');

subplot(512)
plot(tvec,ey(2,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\tilde{e}_{y,\rho_{ga}}$','Interpreter','latex');

subplot(513)
plot(tvec,ey(3,:),'LineWidth',1.5)
grid on; hold on;
```

```
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\tilde{e}_{y,\gamma_{g,a}}$','Interpreter','latex');

subplot(514)
plot(tvec,ey(4,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\tilde{e}_{y,\xi_a}$','Interpreter','latex');

subplot(515)
plot(tvec,ey(5,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\tilde{e}_{y,\eta_a}$','Interpreter','latex');

%% Filter testing LKF
%LKF
%%%***Specify assumed DT noise covariances for KF:
sf = 0.075;
% Qkf=diag(1000*[.01 .01 1 .05 .05 1])*Q; %%treat this as the main tuning knob
% Rkf = .075*R; %%assume we can get this from sensor specs

Qkf=diag(950*[.01 .01 5 .1 .1 5])*Q; %%treat this as the main tuning knob
Rkf = diag([.5 1 .5 1 1])*R*.0625; %%assume we can get this from sensor specs
Qkf = Qkf*sf;

tspan=0:.1:100;
Nsimruns = 50;
NEESsamps = zeros(Nsimruns,length(tvec));
NISsamps = zeros(Nsimruns,length(tvec));

% Generate Offline Nominal Trajectory from Nonlinear Model Dynamics
[t,xnom] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan,x0);
xnom(:,3) = wrapToPi(xnom(:,3));
xnom(:,6) = wrapToPi(xnom(:,6));
xnom = xnom';

ynom = nl_sensor_model(xnom);
ynom(1,:)=wrapToPi(ynom(1,:));
ynom(3,:)=wrapToPi(ynom(3,:));

for ss=1:Nsimruns
    %%%1. %%Generate true trajectory and measurements from system

    xk_truehist = [];
    ykhist = [];
    xk = x0; %sample initial robot state
    [t,xk_truehist] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan,x0);
    xk_truehist = addprocessNoise(xk_truehist',Q);
    xk_truehist=xk_truehist';
    xk_truehist(:,3) = wrapToPi(xk_truehist(:,3));
    xk_truehist(:,6) = wrapToPi(xk_truehist(:,6));
    ynoise=[];

    ynoise=nl_sensor_model_noise(xk_truehist',R);
```

```
    ykhist=ynoise';
    ykhist(1,:)=[];
    ykhist=ykhist';
    ykhist(1,:)=wrapToPi(ykhist(1,:));
    ykhist(3,:)=wrapToPi(ykhist(3,:));
    %%% 2. LKF time

    %%Run the Kalman filter updates
    mk = x0;
    Pk = 9*diag([2 2 2 10 10 2]);
    mk_filt_hist = [];
    Pk_filt_hist = [];
    innovk_hist = [];
    Pyyk_hist = []; %%store measurement innovation covar
    NEESsshist = [];
    NISsshist = [];
    NEESsamps=[];
    NISsamps=[];
    mk_filt_hist(:,1)=mk;
    Pk_filt_hist(:,1)=reshape(Pk,6^2,1);
    delu=[0;0;0;0];
    dy = [];
    dxhat_p=[0;0;0;0;0;0];
    % LKF Loop
    for k = 1:length(ykhist)

        % Prediction Step
        [~,~,F_til,~,H_til] = matrixUpdate(xnom(:,k),u0,dt,L);
        dxhat_m = F_til*dxhat_p;
        dxhat_m(3)=wrapToPi(dxhat_m(3));
        dxhat_m(6)=wrapToPi(dxhat_m(6));
        Pm = F_til*reshape(Pk_filt_hist(:,k),6,6)*F_til' + Omega*Qkf*Omega';

        % Correction Step
        [~,~,F_til,G_til,H_til] = matrixUpdate(xnom(:,k+1),u0,dt,L);
        K = Pm*H_til'*inv(H_til*Pm*H_til' + Rkf);
        Pyykp1 = H_til*Pm*H_til' + Rkf;
        Pyykp1 = 0.5*(Pyykp1 + Pyykp1');

        dy = ykhist(:,k)-ynom(:,k+1);
        dy(1) = wrapToPi(dy(1)); dy(3) = wrapToPi(dy(3));

        innov_kp1 = ykhist(:,k) - H_til*mk_filt_hist(:,k);

        innov_kp1(1) = wrapToPi(innov_kp1(1)); innov_kp1(3) = wrapToPi(
    innov_kp1(3));

        dxhat_p = dxhat_m + K*(dy - H_til*dxhat_m);
        dxhat_p(3)=wrapToPi(dxhat_p(3));
        dxhat_p(6)=wrapToPi(dxhat_p(6));
        Pp = (eye(6) - K*H_til)*Pm;
        Pk_filt_hist(:,k+1) = reshape(Pp,6^2,1);
        mk_filt_hist(:,k+1) = xnom(:,k+1)+dxhat_p;

        mk_filt_hist(3,:) = wrapToPi(mk_filt_hist(3,:));
```

```
        mk_filt_hist(6,:) = wrapToPi(mk_filt_hist(6,:));

        %%Compute and store NEES and NIS statistics:
        invPkp1 = inv(Pp);
        invPyykp1 = inv(Pyykp1);
        neesSub = (xk_truehist(k,:)' - mk_filt_hist(:,k));
        neesSub(3) = wrapToPi(neesSub(3)); neesSub(6) = wrapToPi(neesSub(6));
        NEESsshist(k) = ...
            (xk_truehist(k,:)' - mk_filt_hist(:,k))'*invPkp1*(xk_truehist(k,:)
    ' - mk_filt_hist(:,k));
%         NEESsshist(k) = ...
%             neesSub'*invPkp1*neesSub;
        NISsshist(k) = innov_kp1'*invPyykp1*innov_kp1;
    end


    NEESsamps(ss,:) = NEESsshist;
    NISsamps(ss,:) = NISsshist;
     %%sanity check:
%     mk_filt_hist(3,:) = wrapToPi(mk_filt_hist(3,:));
%     mk_filt_hist(6,:) = wrapToPi(mk_filt_hist(6,:));

    %Plot state estimation errors versus time
    figure(f),
    clf
    subplot(611)
    plot(t,xk_truehist(:,1)-mk_filt_hist(1,:)','LineWidth',2), hold on
    plot(t,2*sqrt((Pk_filt_hist(1,:))),'m--','LineWidth',1)
    plot(t,-2*sqrt((Pk_filt_hist(1,:))),'m--','LineWidth',1)
    subplot(612)
    plot(t,xk_truehist(:,2)-mk_filt_hist(2,:)','LineWidth',2), hold on
    plot(t,2*sqrt((Pk_filt_hist(8,:))),'m--','LineWidth',1)
    plot(t,-2*sqrt((Pk_filt_hist(8,:))),'m--','LineWidth',1)
    subplot(613)
    plot(t,xk_truehist(:,3)-mk_filt_hist(3,:)','LineWidth',2), hold on
    plot(t,2*sqrt((Pk_filt_hist(15,:))),'m--','LineWidth',1)
    plot(t,-2*sqrt((Pk_filt_hist(15,:))),'m--','LineWidth',1)
    subplot(614)
    plot(t,xk_truehist(:,4)-mk_filt_hist(4,:)','LineWidth',2), hold on
    plot(t,2*sqrt((Pk_filt_hist(22,:))),'m--','LineWidth',1)
    plot(t,-2*sqrt((Pk_filt_hist(22,:))),'m--','LineWidth',1)
    subplot(615)
    plot(t,xk_truehist(:,5)-mk_filt_hist(5,:)','LineWidth',2), hold on
    plot(t,2*sqrt((Pk_filt_hist(29,:))),'m--','LineWidth',1)
    plot(t,-2*sqrt((Pk_filt_hist(29,:))),'m--','LineWidth',1)
    subplot(616)
    plot(t,xk_truehist(:,6)-mk_filt_hist(6,:)','LineWidth',2), hold on
    plot(t,2*sqrt((Pk_filt_hist(36,:))),'m--','LineWidth',1)
    plot(t,-2*sqrt((Pk_filt_hist(36,:))),'m--','LineWidth',1)
end

% NEES TEST:
epsNEESbar = mean(NEESsamps,1);
alphaNEES = 0.01; %%significance level
Nnx = Nsimruns*length(F_til);
```

```
%%compute intervals:
r1x = chi2inv(alphaNEES/2, Nnx )./ Nsimruns
r2x = chi2inv(1-alphaNEES/2, Nnx )./ Nsimruns

clf;
f = f+1;
figure(f)
f = f+1;
plot(epsNEESbar,'mo','MarkerSize',6,'LineWidth',2),hold on
plot(r1x*ones(size(epsNEESbar)),'b--','LineWidth',2)
plot(r2x*ones(size(epsNEESbar)),'b--','LineWidth',2)
ylabel('$NEES \; Statistic, \; \bar{\epsilon}_{x}$','FontSize',14,'Interpreter','latex')
xlabel('$time \; step, \; k$','FontSize',14,'Interpreter','latex')
title('$NEES \; Estimation \; Results$','FontSize',14,'Interpreter','latex')
legend('$NEES \; @ \; time \; k$', '$r_1 bound$', '$r_2 bound$','Interpreter','latex')
axis([0 1000 r1x-5 r2x+5])

% NIS TEST:
epsNISbar = mean(NISsamps,1);
alphaNIS = 0.01;
Nny = Nsimruns*size(H_til,1);

% compute intervals:
r1y = chi2inv(alphaNIS/2, Nny )./ Nsimruns
r2y = chi2inv(1-alphaNIS/2, Nny )./ Nsimruns
figure(f)
f = f+1;
plot(epsNISbar,'bo','MarkerSize',6,'LineWidth',2),hold on
plot(r1y*ones(size(epsNISbar)),'m--','LineWidth',2)
plot(r2y*ones(size(epsNISbar)),'m--','LineWidth',2)
ylabel('$NIS \; Statistic, \; \bar{\epsilon}_y$','FontSize',14,'Interpreter','latex')
xlabel('$time \; step,\; k$','FontSize',14,'Interpreter','latex')
title('$NIS \; Estimation \; Results$','FontSize',14,'Interpreter','latex')
legend('$NIS \; @ \; time k$', '$r_1 bound$', '$r_2 bound$','Interpreter','latex')
axis([0 1000 0 15])

%% EKF TESTING
%%%***Specify assumed DT noise covariances for KF:
Qkf=diag([.04 .04 .9 .04 .04 .9])*Q;%diag([10 10 .02 1 1 .02])*Q;  %%treat this as the main tuning knob
Rkf = .06*R; %%assume we can get this from sensor specs
tspan=0:.1:100;
Nsimruns = 50;
NEESsamps = zeros(Nsimruns,length(tvec));
NISsamps = zeros(Nsimruns,length(tvec));

% Generate Offline Nominal Trajectory from Nonlinear Model Dynamics
[t,xnom] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan,x0);
xnom(:,3) = wrapToPi(xnom(:,3));
xnom(:,6) = wrapToPi(xnom(:,6));
```

```
xnom = xnom';

ynom = nl_sensor_model(xnom);
f=f+1;
for ss=1:Nsimruns
    %%%1. %%Generate true trajectory and measurements from system

    xk_truehist = [];
    ykhist = [];
    xk = x0; %sample initial robot state
    [t,xk_truehist] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan,x0);
    xk_truehist = addprocessNoise(xk_truehist',Q);
    xk_truehist=xk_truehist';
    xk_truehist(:,3) = wrapToPi(xk_truehist(:,3));
    xk_truehist(:,6) = wrapToPi(xk_truehist(:,6));
    ynoise=[];

  ynoise=nl_sensor_model_noise(xk_truehist',R);
    ykhist=ynoise';
    ykhist(1,:)=[];
    ykhist=ykhist';
    %EKF time
    mk = x0;
    Pk = diag([64 64 4 170 170 4]);
    mk_filt_hist = [];
    Pk_filt_hist = [];
    innovk_hist = [];
    Pyyk_hist = []; %%store measurement innovation covar
    NEESsshist = [];
    NISsshist = [];
    NEESsamps=[];
    NISsamps=[];
    mk_filt_hist(:,1)=mk;
    Pk_filt_hist(:,1)=reshape(Pk,6^2,1);
    delu=[0;0;0;0];
    dy = [];
    tspan2 = 0:.1:.1;
    %EKF look
    for k=1:length(ykhist)
% Prediction Step
        [~,~,F_til,~,~] = matrixUpdate(xnom(:,k),u0,dt,L);
        [t,xnew] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan2,
    mk_filt_hist(:,k));
        xhat_m2 = xnew(end,:)';
        %xhat_m2(3)=wrapToPi(xhat_m2(3));
        %xhat_m2(6)=wrapToPi(xhat_m2(6));
        Pm = F_til*reshape(Pk_filt_hist(:,k),6,6)*F_til' + Omega*Qkf*Omega';
% Correction Step
         [~,~,~,~,H_til] = matrixUpdate(xhat_m2,u0,dt,L);
          yhat_m = nl_sensor_model(xhat_m2);
          %yhat_m(1)=wrapToPi(yhat_m(1));
          %yhat_m(3)=wrapToPi(yhat_m(3));
          ey = ykhist(:,k) - yhat_m;
          ey(1)=wrapToPi(ey(1));
          ey(3)=wrapToPi(ey(3));
```

```
            innov_kp1 = ey;
            innov_kp1(1) = wrapToPi(innov_kp1(1)); innov_kp1(3) = wrapToPi(innov_kp1(3));

            K = Pm*H_til'*inv(H_til*Pm*H_til' + Rkf);
            xhat_p2 = xhat_m2 + K*ey;
            xhat_p2(3)=wrapToPi(xhat_p2(3));
            xhat_p2(6)=wrapToPi(xhat_p2(6));
            mk_filt_hist(:,k+1) = xhat_p2;
            Pp = (eye(6) - K*H_til)*Pm;
            Pk_filt_hist(:,k+1) = reshape(Pp,6^2,1);
            Pyykp1 = H_til*Pm*H_til' + Rkf;
            Pyykp1 = 0.5*(Pyykp1 + Pyykp1');
%%Compute and store NEES and NIS statistics:
          invPkp1 = inv(Pp);
          invPyykp1 = inv(Pyykp1);
          NEESsshist(k) = ...
              (xk_truehist(k,:)' - mk_filt_hist(:,k))'*invPkp1*(xk_truehist(k,:)' - mk_filt_hist(:,k));
          NISsshist(k) = innov_kp1'*invPyykp1*innov_kp1;
      end
      NEESsamps(ss,:) = NEESsshist;
      NISsamps(ss,:) = NISsshist;
       %%sanity check:
%       mk_filt_hist(3,:) = wrapToPi(mk_filt_hist(3,:));
%       mk_filt_hist(6,:) = wrapToPi(mk_filt_hist(6,:));

      %Plot state estimation errors versus time
      figure(f),
      clf
      subplot(611)
      plot(tvec,xk_truehist(:,1)-mk_filt_hist(1,:)','b','LineWidth',2), hold on
      plot(tvec,2*sqrt((Pk_filt_hist(1,:))),'b--','LineWidth',1)
      plot(tvec,-2*sqrt((Pk_filt_hist(1,:))),'b--','LineWidth',1)
      subplot(612)
      plot(tvec,xk_truehist(:,2)-mk_filt_hist(2,:)','b','LineWidth',2), hold on
      plot(tvec,2*sqrt((Pk_filt_hist(8,:))),'b--','LineWidth',1)
      plot(tvec,-2*sqrt((Pk_filt_hist(8,:))),'b--','LineWidth',1)
      subplot(613)
      plot(tvec,xk_truehist(:,3)-mk_filt_hist(3,:)','b','LineWidth',2), hold on
      plot(tvec,2*sqrt((Pk_filt_hist(15,:))),'b--','LineWidth',1)
      plot(tvec,-2*sqrt((Pk_filt_hist(15,:))),'b--','LineWidth',1)
      subplot(614)
      plot(tvec,xk_truehist(:,4)-mk_filt_hist(4,:)','b','LineWidth',2), hold on
      plot(tvec,2*sqrt((Pk_filt_hist(22,:))),'b--','LineWidth',1)
      plot(tvec,-2*sqrt((Pk_filt_hist(22,:))),'b--','LineWidth',1)
      subplot(615)
      plot(tvec,xk_truehist(:,5)-mk_filt_hist(5,:)','b','LineWidth',2), hold on
      plot(tvec,2*sqrt((Pk_filt_hist(29,:))),'b--','LineWidth',1)
      plot(tvec,-2*sqrt((Pk_filt_hist(29,:))),'b--','LineWidth',1)
      subplot(616)
      plot(tvec,xk_truehist(:,6)-mk_filt_hist(6,:)','b','LineWidth',2), hold on
      plot(tvec,2*sqrt((Pk_filt_hist(36,:))),'b--','LineWidth',1)
      plot(tvec,-2*sqrt((Pk_filt_hist(36,:))),'b--','LineWidth',1)
end
```

```
%%DO NEES TEST:
epsNEESbar = mean(NEESsamps,1);
alphaNEES = 0.05; %%significance level
Nnx = Nsimruns*length(F_til);
%%compute intervals:
r1x = chi2inv(alphaNEES/2, Nnx )./ Nsimruns
r2x = chi2inv(1-alphaNEES/2, Nnx )./ Nsimruns

f=f+1;
figure(f)
plot(epsNEESbar,'mo','MarkerSize',6,'LineWidth',2),hold on
plot(r1x*ones(size(epsNEESbar)),'b--','LineWidth',2)
plot(r2x*ones(size(epsNEESbar)),'b--','LineWidth',2)
ylabel('NEES statistic, \bar{\epsilon}_x','FontSize',14,'Interpreter',tex)
xlabel('time step, k','FontSize',14)
title('NEES Estimation Results','FontSize',14)
legend('NEES @ time k', 'r_1 bound', 'r_2 bound')
axis([0 1000 0 20])
%%DO NIS TEST:
epsNISbar = mean(NISsamps,1);
alphaNIS = 0.05;
Nny = Nsimruns*size(H_til,1);
%%compute intervals:
r1y = chi2inv(alphaNIS/2, Nny )./ Nsimruns
r2y = chi2inv(1-alphaNIS/2, Nny )./ Nsimruns
f=f+1;
figure(f)
plot(epsNISbar,'bo','MarkerSize',6,'LineWidth',2),hold on
plot(r1y*ones(size(epsNISbar)),'m--','LineWidth',2)
plot(r2y*ones(size(epsNISbar)),'m--','LineWidth',2)
ylabel('NIS statistic, \bar{\epsilon}_y','FontSize',14)
xlabel('time step, k','FontSize',14)
title('NIS Estimation Results','FontSize',14)
legend('NIS @ time k', 'r_1 bound', 'r_2 bound')
axis([0 1000 0 20])


%% Function Definitions

function xdot_nl = x_nl_traject(x,L,u,u0,t)
% Combined UGV & UAV Nonlinear Dynamics with control forcing input
u = u(t)+u0;

xdot_nl = [u(1)*cos(wrapToPi(x(3))); u(1)*sin(wrapToPi(x(3))); (u(1)/L)*tan(u(2));...
    u(3)*cos(wrapToPi(x(6))); u(3)*sin(wrapToPi(x(6))); u(4)];
end

function xdot_nl_check = x_nl_traject_check(x,L,dx,u)
% Combined UGV & UAV Nonlinear Dynamics with x-perturbation input
delx = [dx(1); dx(2); dx(3); dx(4); dx(5); dx(6)];

xdot_nl = [u(1)*cos(wrapToPi(x(3))); u(1)*sin(wrapToPi(x(3))); (u(1)/L)*tan(u(2));...
    u(3)*cos(wrapToPi(x(6))); u(3)*sin(wrapToPi(x(6))); u(4)] + delx;
```

```
xdot_nl_check = [xdot_nl(1); xdot_nl(2); wrapToPi(xdot_nl(3));...
    xdot_nl(4); xdot_nl(5); wrapToPi(xdot_nl(6))];
end

function y_obs = nl_sensor_model(x)
% Nonlinear Sensor Dynamics Model

y_obs(1,:) = wrapToPi(atan2((x(5,:)-x(2,:)),(x(4,:)-x(1,:)))-x(3,:));
y_obs(2,:) = sqrt((x(1,:)-x(4,:)).^2+(x(2,:)-x(5,:)).^2);
y_obs(3,:) = wrapToPi(atan2((x(2,:)-x(5,:)),(x(1,:)-x(4,:)))-x(6,:));
y_obs(4,:) = x(4,:);
y_obs(5,:) = x(5,:);

end

function y_obs = nl_sensor_model_noise(x,R)
% Nonlinear Sensor Dynamics Model

y_obs(1,:) = wrapToPi(atan2((x(5,:)-x(2,:)),(x(4,:)-x(1,:)))-x(3,:))+sqrt(R
    (1,1))*randn(1,length(x));
y_obs(2,:) = sqrt((x(1,:)-x(4,:)).^2+(x(2,:)-x(5,:)).^2)+sqrt(R(2,2))*randn(1,
    length(x));
y_obs(3,:) = wrapToPi(atan2((x(2,:)-x(5,:)),(x(1,:)-x(4,:)))-x(6,:))+sqrt(R
    (3,3))*randn(1,length(x));
y_obs(4,:) = x(4,:)+sqrt(R(4,4))*randn(1,length(x));
y_obs(5,:) = x(5,:)+sqrt(R(5,5))*randn(1,length(x));

end

function [A,B,F,G,H] = matrixUpdate(x,u,dt,L)
% Update Linearized CT Matrices and Discretize

    % A: Linearized Plant Dynamics Matrix
    A = [0 0 -u(1)*sin(x(3)) 0 0 0;...
        0 0 u(1)*cos(x(3)) 0 0 0;
        0 0 0 0 0 0;...
        0 0 0 0 0 -u(3)*sin(x(6));...
        0 0 0 0 0 u(3)*cos(x(6));...
        0 0 0 0 0 0];

    % B: Linearized Control Input Matrix
    B = [cos(x(3)) 0 0 0;...
        sin(x(3)) 0 0 0;...
        (tan(u(2))/L) ((u(1)/L)*(sec(u(2))^2)) 0 0;...
        0 0 cos(x(6)) 0;...
        0 0 sin(x(6)) 0;...
        0 0 0 1];

    % H: Linearized Sensor Matrix
    H(1,1) = (x(5)-x(2))/(((x(5)-x(2))^2/(x(4)-x(1))^2+1)*(x(4)-x(1))^2);
    H(1,2) = -1/(((x(5)-x(2))^2/(x(4)-x(1))^2+1)*(x(4)-x(1)));
    H(1,3) = -1;
    H(1,4) = -H(1,1);
    H(1,5) = -H(1,2);
```

```
    H(1,6) = 0;
    H(2,1) = (x(1)-x(4))/sqrt((x(1)-x(4))^2+(x(2)-x(5))^2);
    H(2,2) = (x(2)-x(5))/sqrt((x(1)-x(4))^2+(x(2)-x(5))^2);
    H(2,4) = -H(2,1);
    H(2,5) = -H(2,2);
    H(3,1) = -(x(2)-x(5))/(((x(2)-x(5))^2/(x(1)-x(4))^2+1)*(x(1)-x(4))^2);
    H(3,2) = 1/(((x(2)-x(5))^2/(x(1)-x(4))^2+1)*(x(1)-x(4)));
    H(3,4) = -H(3,1);
    H(3,5) = -H(3,2);
    H(3,6) = -1;
    H(4,4) = 1;
    H(5,5) = 1;

    % F & G: Linearized & Discretized State Matrices
    F = eye(6) + dt*A;
    G = dt*B;
end
function xnoise = addprocessNoise(X,Q)
for i=1:length(X)
    X(1,i)=X(1,i)+1*sqrt(Q(1,1))*randn(1);
    X(2,i)=X(2,i)+1*sqrt(Q(2,2))*randn(1);
    X(3,i)=X(3,i)+1*sqrt(Q(3,3))*randn(1);
    X(4,i)=X(4,i)+1*sqrt(Q(4,4))*randn(1);
    X(5,i)=X(5,i)+1*sqrt(Q(5,5))*randn(1);
    X(6,i)=X(6,i)+1*sqrt(Q(6,6))*randn(1);
end
xnoise=X;
end
```